

AgentX：迈向工业推荐系统的智能体驱动自迭代

AgentX Team

推荐算法迭代正从依赖工程师的手工作坊式流程迈向工业化研究闭环，但这一转型仍受制于结构性的执行瓶颈：从想法到上线的周期依然依赖人类工程师提出假设、修改生产代码、发起 A/B 实验并归因线上结果。因此，创新能力只能随人力投入线性增长，而无法随证据、算力和累积的实验知识产生复利效应。我们提出 AgentX，这是一个已部署于生产环境的多智能体系统，从根本上重构了这一生产函数。AgentX 作为一个自进化开发引擎运行：它以任何人工工作流都无法持续维持的规模与速度，自主生成、实现、评估推荐实验并从中学习。

该系统以闭环方式编排四个紧密耦合的阶段。Brainstorm Agent 将历史实验、系统架构、数据分析和外部研究中的证据综合为经过排序且可执行的提案。Developing Agent 通过基于代码仓库的生成和多维可靠性验证，将每项提案转化为生产就绪代码。Evaluation Agent 以护栏否决式 A/B 判定执行安全的线上发布，并将成功与失败均转化为结构化知识资产。随后，Harness Evolution 层（SGPO）将执行轨迹提炼为语义梯度更新，持续增强智能体自身能力—使系统不仅实现自动化，更能自我改进。

在为期三周、覆盖快手 App 主信息流和生活服务推荐的部署中，三个 AgentX worker 将 374 个想法转化为 10 个可上线发布项—借助自进化，每个 worker 的吞吐量每周翻倍，相较一名人工工程师实现了 8× 的并发能力和 3.7× 的业务价值，带来 0.561% 的用户 App 使用时长提升，以及超过人民币 1 亿元的年化收入。除线上策略迭代外，同一闭环原则还扩展到模型侧研究，支持自主论文复现、模块消融和跨论文架构组合。这些结果表明，自进化、批量规模的智能体驱动迭代已不再是假设—它如今已能在生产推荐系统中带来复利回报。

AgentX 指向推荐研究与开发的一种新分工：一层工程师与智能体系统协作，大规模加速策略和模型迭代；另一层则演进智能体框架及其基础模型，持续放大系统能力。每次实验产生的轨迹数据同时为二者提供养料—将日常执行与长期智能增长连接起来。人力的线性叠加首次可以被改写为可复利的杠杆效应。



Contents

1 引言	5
2 相关工作	7
2.1 LLM 驱动的 AutoML	7
2.2 面向 RecSys 的智能体自动化	7
3 多智能体设计框架	8
4 头脑风暴智能体	8
4.1 歧义问题	10
4.2 有界提案探索	10
4.3 证据加权的提案生成	11
4.3.1 Experiment KB	11
4.3.2 System KB	12
4.3.3 Data Analysis	12
4.3.4 Model Research	12
4.4 验证与实现交付	13
5 开发智能体	14
5.1 线上策略开发	14
5.1.1 生产代码可靠性问题	14
5.1.2 仓库依据的代码生成	14
5.1.3 面向验证的实现循环	15
5.1.4 质量评分	16
5.1.5 失败模式	16
5.2 模型开发	17
5.2.1 单轮流水线	17
5.2.2 可证伪归因	18

5.2.3 平台故障下的稳健执行	18
6 评估智能体	19
6.1 真实世界奖励问题	19
6.2 安全部署与流量分配	20
6.3 带护栏否决的 A/B 判定	20
6.4 负面结果资产化	21
7 AgentX 框架演化	21
7.1 基于语义梯度的提示词优化	22
7.1.1 基于语义梯度的提示词优化 I	22
7.1.2 基于语义梯度的提示词优化 II	25
7.2 演进模型研究探索流水线	26
7.3 讨论	28
8 实验	28
8.1 生产测量与闭环性能	28
8.2 AgentX 的模型迭代漏斗	32
8.3 案例展示 I: 端到端 AgentX 实验	33
8.4 案例展示 II: 与专家智能体协同演进	34
9 结论	36
A 作者名单	42
B 模型研究实验	42
B.1 复现实验	42
B.2 模块探索实验	43
B.3 工业场景实验	45
C 提示词格式	45

C.1 Brainstorm 提示词	45
C.2 开发提示词	48
C.3 评估提示词	49

1. 引言

工业推荐系统 (Guo et al., 2017; Zhou et al., 2018; Deng et al., 2025; Zhu et al., 2025) 的迭代长期依赖算法工程师持续参与。一次完整的算法迭代通常要经历数据分析、特征工程、模型开发、线上部署、A/B 测试和归因复盘等多个阶段，并依赖异构数据平台、训练框架和监控工具，发布周期往往以周计。工业推荐系统面对复杂业务目标和严格工程约束，因此有效的系统改进高度依赖难以显式编码的专家经验。整个系统的迭代吞吐量在很大程度上受工程师数量及其认知负荷限制。

然而，工程师的大量精力并未用于高价值判断，而是分散在拉取数据、修改配置、维护流水线等重复性事务上。在显著的工程摩擦下，这稀释了真正稀缺的能力，即提出高质量假设、理解实验失败原因以及识别系统优化方向。无论增加工程师人数，还是建设更强的底层平台，传统补救方法本质上都只是对人工开发流程的线性放大，未能改变推荐系统迭代高度依赖人力和个体经验的结构瓶颈。

随着大语言模型 (LLM) (Achiam et al., 2023; Liu et al., 2024; Team et al., 2024; Yang et al., 2025a) 在代码理解、多步推理、错误诊断和工具调用方面不断进步，以智能体作为执行主体来驱动算法迭代已开始具备实践可行性。智能体 (Team et al., 2025; Zeng et al., 2026) 不仅能够理解特征定义、训练脚本和实验日志等完整工程上下文，还能通过修改代码和调用内部平台直接作用于这些上下文，从而转移推荐系统迭代的核心瓶颈：

从算法工程师的人力转向智能体系统的能力。

前者高度依赖个体经验，后者则可复现、可复利。一旦每次线上迭代的执行轨迹都能被记录、分析并反馈到底层智能体框架，系统本身就不再只是线性自动化工具，而会成为随时间不断增强的开发机制，使构建自动化开发闭环在实践中成为可能。

相应地，近年来，**LLM 驱动的机器学习 (ML) 开发自动化**涌现出一系列代表性工作，可按逐级提高的自主性分为三个层次。第一层是**流水线自动化**，将 LLM 集成到预定义的 AutoML (Thornton et al., 2013) 和数据科学工作流中，而任务定义和总体解空间仍主要由人类指定 (Hollmann et al., 2023)。第二层是**实验自动化**，不再局限于执行既有流水线，而是允许智能体探索实现和实验策略，并依据实证反馈改进决策 (Tiriat et al., 2024; Nam et al., 2026)。第三层是**研究自动化**，进一步将自主性扩展到研究的提出和验证，对问题、假设、方法和证据进行联合探索 (Lu et al., 2024; Schmidgall et al., 2025)。这些工作共同推动机器学习开发从单阶段辅助迈向融合工程执行、实验迭代和知识发现的智能体自动化。

然而，从工业推荐场景审视这些工作，可以发现三类共同的结构缺口：

- **缺少真实线上反馈**。现有系统采用的成功信号主要来自离线指标或人类专家评分。这些信号用于学术评估尚属合理，却并不等同于对真实业务价值的正向贡献 (Fabijan et al., 2018)。线上 A/B 结果是与真实价值最一致且最不易受噪声影响的奖励信号 (Xu et al., 2015; Zhang et al., 2026b)，但在现有研究中基本缺席。

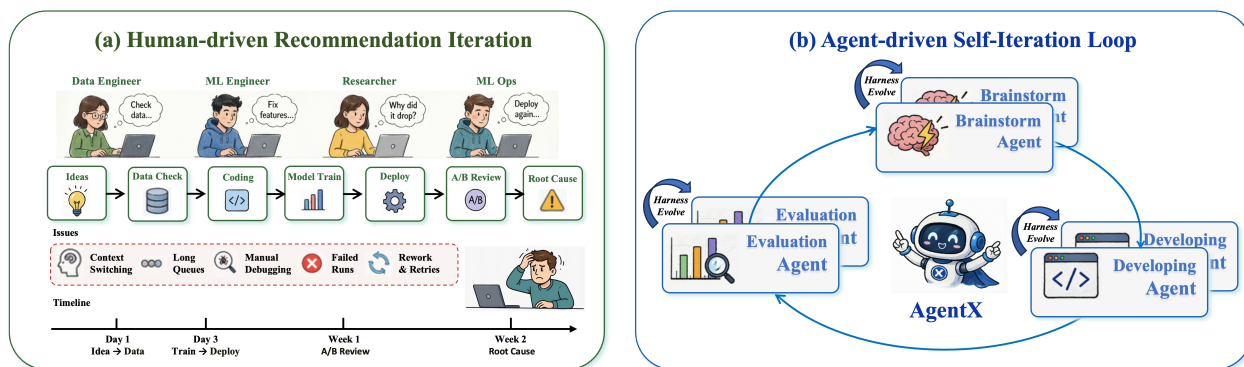


Figure 1 | AgentX 将推荐迭代从人类驱动、人工交接的流水线转化为智能体驱动的闭环，线上 A/B 反馈和轨迹数据在其中持续改进系统本身。

- **工业规模验证有限。**当前公开工作所验证的数据集、模型和流水线规模仍落后于真实推荐系统；后者涉及十亿级样本、耦合的多目标优化，以及多条业务线并行运行 (Cao et al., 2024; Wang et al., 2026c)。这种规模差异不仅影响绝对性能，还会从根本上重塑系统的瓶颈结构：在小规模下可忽略的工程约束，会成为决定闭环在工业规模下能否成立的关键变量。
- **缺少持续演化机制。**多数现有 ML 工程智能体系统在完成单项任务后不会积累系统的参数、提示词和工作流，因而无法产生长期复利效应。系统因此难以适应新任务和变化的环境，反复陷入相同陷阱而不能复用工程经验，限制了系统智能水平的持续提升。

换言之，现有工作已经回答了智能体能否在受控环境中完成 ML 任务，但尚未解决一个对业务更为关键的问题：智能体能否在真实工业推荐系统中持续带来线上增益，并随时间自我演化。一旦智能体使想法的生成和验证不再天然稀缺，关键便转向如何构建适配真实业务场景的闭环、全生命周期开发流水线。

基于上述背景，我们提出并部署了面向工业推荐系统的自动化开发框架 **AgentX**。AgentX 的目标不是让 LLM 在沙箱中完成一次性 ML 任务，而是持续推动真实推荐业务中的算法迭代。具体而言，AgentX 构成完整的在线反馈闭环，涵盖想法生成、代码与流水线开发、A/B 分析反馈以及下一轮推演。每次执行过程都以轨迹形式持续沉淀，作为后续优化智能体工作流、训练基础模型和改进系统策略的数据源。因此，AgentX 不仅是自动化人工步骤的工具，更是具备自我迭代能力的推荐开发系统。

本文主要贡献如下：

- **完整闭环。**我们在真实工业推荐场景中打通了从调研到生产部署的端到端算法开发与演化流水线。系统不依赖人工拼接每个步骤，而是由智能体在统一编排下持续驱动流程。
- **真实工业奖励。**AgentX 的最终成功标准以真实在线 A/B 反馈为依据，利用来自用户的最干净训练信号，面向工业场景的真实需求，并使整个系统的优化目标直接对齐真实业务收益。
- **可扩展收益与自我迭代飞轮。**AgentX 能够并发生成多个想法，并通过统一质量漏斗加以验证，从而提升实验吞吐量和在线收益规模。同时，每个实验产生的轨迹都会反馈至智能体编排框架

和基础模型，使系统在长期运行中不断增强。

下文首先明确 AgentX 与相关工作的区别，继而详述这一自动化开发闭环系统的设计。随后展示基于轨迹的自我迭代飞轮，以及实验结果和规模化真实业务收益。最后总结 AgentX 如何重塑现有工业推荐系统的迭代范式。

2. 相关工作

2.1. LLM 驱动的 AutoML

传统 AutoML (Thornton et al., 2013) 将机器学习开发表述为在人类定义的模型、流水线和超参数空间上的优化。LLM 通过语义推理、可执行代码生成以及与开发环境交互扩展了这一范式。近期工作可分为三个自动化程度递进的层次。

第一层是**流水线自动化**，将 LLM 引入结构化 AutoML，但任务定义、评估目标和总体开发流程仍主要由人指定。CAAFE (Hollmann et al., 2023) 利用上下文知识生成具有语义意义的特征；DS-Agent (Guo et al., 2024) 和 Data Interpreter (Hong et al., 2025) 则结合规划、检索和代码执行，实现多阶段数据分析自动化。AutoML-Agent (Trirat et al., 2024) 进一步在统一工作流中协调数据处理、特征工程、模型构建、训练和验证。尽管覆盖广泛，这些方法主要旨在实例化并执行可识别的机器学习流水线，而非自主确定长期实验策略。

第二层是**实验自动化**，将可执行实现和实验轨迹视为自主搜索对象。机器学习工程智能体反复修改代码、执行实验、解读指标与日志，并根据经验反馈选择后续行动。IMPROVE (Xue et al., 2025) 和 MLE-STAR (Nam et al., 2026) 基于归因或消融分析改进选定的流水线组件，AIDE (Trirat et al., 2024) 通过结构化搜索维护多个候选方案。R&D-Agent (Yang et al., 2025b) 将研究推理与实现分离，AutoMind (Ou et al., 2025) 和 MLEvolve (Du et al., 2026) 则利用记忆或技能学习跨实验积累经验。

第三层是**研究自动化**，将决策空间从求解预定义任务进一步扩展到提出并验证研究本身。The AI Scientist (Lu et al., 2024) 和 Agent Laboratory (Schmidgall et al., 2025) 在统一工作流中整合假设生成、方法开发、实验和科学写作，更新的系统还引入了对替代假设、实验设计和支持证据的结构化探索 (Yamada et al., 2025; Tang et al., 2026)。研究智能体除提升给定指标外，还必须推理新颖性、证据有效性和可复现性。这三个层次共同反映了从自动化既定开发流程，到自主改进可执行方案，再到探索机器学习研究问题的演进。

2.2. 面向 RecSys 的智能体自动化

推荐系统的智能体自动化关注设计并改进底层推荐模型的智能体，而非直接充当推荐器的 LLM 智能体。早期 AutoML 方法 (Liu et al., 2020; Song et al., 2020; Zhang et al., 2023) 也侧重于在预定义推荐系统框架内对特定组件进行空间搜索。近期智能体方法则超越固定算子，直接针对可执行实现和实验反馈进行推理。

推荐开发的一项独特挑战是，离线排序指标只能不完美地反映线上用户响应。因此，业界对此

方向的探索将进一步把模型修改与生产试验连接起来。Self-EvolveRec (Kim et al., 2026) 利用排序指标和诊断反馈演进推荐代码，以识别并解决模型故障。Self-Evolving (Wang et al., 2026b) Recommendation System 将离线搜索与线上验证结合，AgenticRecTune (Wu et al., 2026) 则利用可复用经验优化多阶段推荐流水线中的配置。A/B Agent (Zhang et al., 2026b) 通过模拟用户交互，提供超越离线排序指标的更丰富评估信号。然而，如何将延迟、非平稳且可能来自模拟的用户反馈可靠地转化为后续模型改进，仍是核心挑战。

3. 多智能体设计框架

在详细介绍各智能体和框架演化方法之前，我们先概述 AgentX 的设计原则：工业推荐系统中，良好的迭代闭环应当是什么样的？在涉及数学、代码或沙盒 ML 基准的经典 LLM 智能体任务中，通常假定环境提供可验证奖励、单条执行轨迹足以判断成功，且优化目标在智能体开始之前已固定。然而，工业推荐并不符合这三项假设：奖励信号存在于延迟的线上 A/B 反馈而非离线指标中；单次部署在任何分数可用之前都要经过护栏和人工审查；优化目标本身还会随业务目标、流量构成和平台约束的演化而漂移。因此，生产级推荐智能体循环必须是闭环，而非一次性执行。具体而言，它需要将模糊业务意图转化为基于证据的提案，将每项提案转化为与仓库一致的代码，通过带护栏否决的安全线上发布验证改动，并将正负轨迹都反馈给系统，使闭环自身随时间改进。这一区别推动了我们的架构与框架演化设计。

我们沿单一闭环将 AgentX 分解为四个阶段：前三个阶段共同推动一个想法从意图走向经验证的线上结果，第四个阶段则反馈所得轨迹以改进闭环本身：

- **Brainstorm Agent**。通过有界探索和证据加权生成，将欠明确的用户意图转化为一小组排序后的可执行实验提案。
- **Developing Agent**。通过基于仓库的生成和面向验证的实现闭环，将每项入选提案转化为生产就绪代码。
- **Evaluation Agent**。管理发布和流量分配，通过护栏否决判断 A/B 结果，并将线上结果资产化为可复用奖励信号和失败记忆。
- **Harness Evolution**。对累积的执行轨迹进行推理，通过基于语义梯度的提示词优化 (SGPO) 更新各子智能体规范，且只有通过配对回放后才予以接纳。

围绕该循环，共享数据层涵盖当前实践经验的知识库，以及记录实验报告的 Agent Data Management；它持久化循环产生的每一项产物，而 Monitoring Platform 则通过仪表盘、指标、追踪、告警、审计和可视化持续观察系统健康状态。Agentx 的总体框架如图 2 所示。

4. 头脑风暴智能体

Brainstorm Agent 是 AgentX 循环的入口：它决定下游智能体可以实现、评估和上线哪些内容。因此，它的职责并非生成一长串看似合理的想法，而是将模糊的优化意图转化为一小组经过排序、可

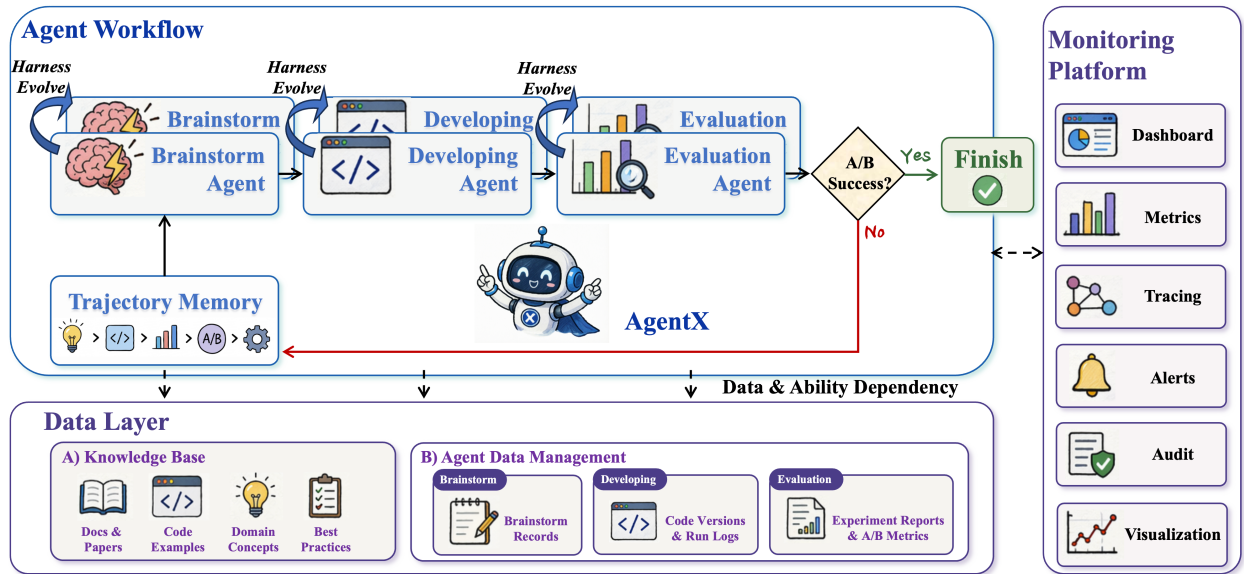


Figure 2 | AgentX 总体框架。

执行的实验提案。每个提案都必须以生产证据为依据，将范围限定在允许的变更面内，并达到足够精确的规格，使其能够被编码、上线并在之后得到诊断。

这一任务很困难，因为工业推荐请求在输入时很少是完整的。用户可能只给出宽泛目标；相关信号可能承载脆弱的业务语义；历史上线评审可能已经包含失败变体；而数据源、流水线阶段或指标路径仍未明确的提案，可能被悄然错误实现，或在上线时遭到拒绝。Brainstorm Agent 通过图 3 所示的证据驱动提案流水线来弥合这一缺口。

该流水线划分了三项职责。

1. **问题界定**。问题模块将定义不足的用户意图转化为明确的任务边界。
2. **想法生成**。生成模块在两项相互耦合的控制下产生候选想法。
 - (a) **有界提案探索**。想法按批次生成，而非作为一次自由形式的回答；每个想法都与目标、可能的实现面、预期下游产物和成熟度状态绑定。
 - (b) **证据上下文**。AgentX 从四类来源检索证据：Experiment KB、System KB、Data Analysis 和 Model Research。Experiment KB 存储历史上线评审与经验；System KB 存储架构、特征、流水线及代码范围知识；Data Analysis 提供基于指标和 SQL 的实证检查；Model Research 将论文机制与生产约束对齐。
3. **验证与物化**。验证模块通过人工评审和生产检查筛选候选项，包括 A/B 参数检查、知识库一致性检查、代码范围检查和边界检查。随后，物化模块将通过的候选项转换为结构化提案产物，以便交付给验证、编码、上线以及后续实验诊断环节。

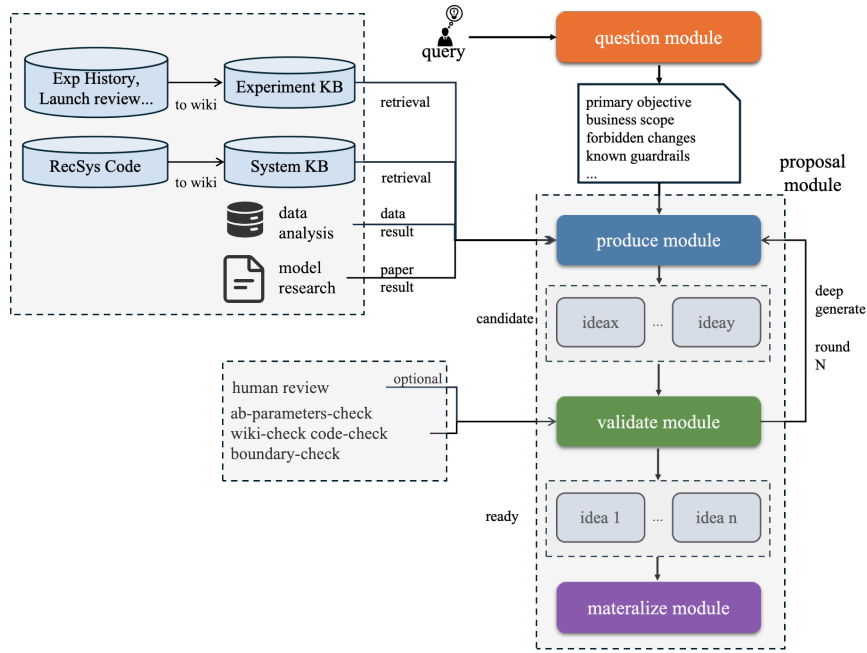


Figure 3 | Brainstorm Agent 工作流。历史实验记忆和系统知识被组织为 Experiment KB 与 System KB。Brainstorm Agent 澄清任务、生成候选想法，通过审查和边界检查进行验证，并将获准想法实体化以供下游执行。

4.1. 歧义问题

头脑风暴的核心难点在于：输入被有意设定为欠明确，而输出却必须在操作层面精准可执行。如果智能体探索得过于自由，就会虚构信号、假设存在实际不可用的特征，或提出超出允许代码路径的改动。如果过于保守，它又只会给出熟悉策略的局部变体，无法拓展机会空间。因此，Brainstorm Agent 被设计为一个边界设定与证据聚合模块：它保留创造性搜索，但将其严格限制在有记录的生产边界内。

对于每项任务，智能体首先将用户请求规范化为一个接入边界。该边界记录主要目标、允许的业务范围、禁止的改动、已知护栏、候选输出要求以及尚未解决的问题。其目的并非强迫用户预先明确所有细节。相反，这一边界旨在显式呈现不确定性。已知字段用于约束搜索；未知字段则转化为保守默认值或具名的后续探查项。这可以防止歧义在未被察觉的情况下传递给 Developing Agent。

4.2. 有界提案探索

边界确定后，Brainstorm Agent 会按批次探索提案，而不是仅生成一次自由形式回答。批次结构十分重要，因为生产想法之间的差异不仅在于机制，也在于成熟度。有些想法可直接实现，有些需要先探查数据或来源，还有些具有战略意义，但依赖未来的平台或模型能力。若将这些情况视为同一种输出，要么会丢弃有价值的长期方向，要么会过早地将不成熟想法推入编码阶段。

因此，AgentX 将每个候选项归入三种状态之一。READY-TO-IMPLEMENT 候选项具有具体目标、推荐流水线中的明确位置、合理的目标达成路径，以及足以进入评审的证据。PROBE-FIRST 候选项

虽有潜力，但实现前需要特定的数据、来源或试运行检查。**MOONSHOT-BACKLOG** 候选项保留可能在未来基础设施、数据或模型改进后变得有用的方向。这种成熟度划分使系统能够广泛探索，同时不混淆探索与承诺。

批次循环还具有残差性。每轮之后，被拒方向、历史重复项、违反的约束以及已覆盖机制都会写入规避集合。有潜力但不完整的方向会转化为明确的探查任务。下一轮随后搜索剩余空间，而不是换一种说法重复相同想法。从这个意义上说，**Brainstorm Agent** 并非只是从 LLM 中采样建议，而是在生产约束下收缩可行机会空间。

4.3. 证据加权的提案生成

如果将所有检索到的上下文视为同等权威，头脑风暴便不可靠。不同候选项需要不同类型的证据。面向新颖性的想法应对照历史上线评审，以避免重复已知失败。对代码路径敏感的想法应对照架构和源代码范围知识，以避免无法实现的交付。指标诊断类想法应以数据分析而非直觉为依据。因此，**Brainstorm Agent** 将知识表示为候选项特定的证据混合，而非固定的检索结果。

令 c 表示候选想法， q 表示结构化的输入边界。证据来源包括以下四类：

$$\mathcal{K} = \{\text{Experiment KB}, \text{System KB}, \text{Data Analysis}, \text{Model Research}\}. \quad (1)$$

随后，加权机制使同一候选项格式能够支持不同推理范式，而无需硬编码单一检索策略：

$$\alpha_k(q, c) \geq 0, \quad \sum_{k \in \mathcal{K}} \alpha_k(q, c) = 1, \quad (2)$$

其中， $\alpha_k(q, c)$ 反映来源 k 对当前问题和候选项的相关性与可靠性。每个来源产生证据分数 $e_k(c) \in [0, 1]$ ，加权证据项为

$$E(c | q) = \sum_{k \in \mathcal{K}} \alpha_k(q, c) e_k(c). \quad (3)$$

最终候选项分数将该证据项与目标一致性、业务有效性、实现可行性、交付完整性和风险相结合：

$$S(c | q) = \lambda_o O(c, q) + \lambda_b B(c) + \lambda_f F(c) + \lambda_h H(c) + \lambda_e E(c | q) - \lambda_r R(c). \quad (4)$$

这里， O 衡量与用户首要目标的一致性， B 衡量业务语义有效性， F 衡量实现可行性， H 衡量交付是否足够完整以供下游执行。风险项 R 惩罚重复方向、未解决的核心信号、过宽范围、不安全的权衡以及其他提案缺陷。

4.3.1. Experiment KB

Experiment KB 存储历史上线评审、业务定义、过往实验结论以及历次迭代中记录的经验。它不仅记录想法成败，还记录结果背后的上下文，如目标场景、受影响用户群、指标变化、上线决策和事后诊断。

该来源主要用于防止 **Brainstorm Agent** 重新发现已知失败或误用业务概念。例如，一个在建模层面看似新颖的候选项，可能已用不同名称测试过，或违反此前上线评审中澄清的业务定义。**Experiment KB** 提供在下游执行前识别此类情况所需的历史依据。

当候选项依赖新颖性、既有实证或业务语义时，Experiment KB 会获得更高权重。它有助于惩罚重复方向、规避已被证伪的假设，并复用成功上线的经验。通过将历史实验记忆纳入候选项评分，Experiment KB 使头脑风暴具有累积性，而非无状态。

4.3.2. System KB

System KB 提供内部推荐系统的结构化知识，包括模型架构、特征定义、DSL 行为、流水线边界、配置语义和源代码范围。其目的是减少 Brainstorm Agent 提出代码路径敏感想法时的幻觉：候选项不仅应在语义上合理，还必须能在实际系统内实现。

System KB 被构建为结构化领域维基，而非扁平文档集合。它将私有系统知识组织为三层：定义各知识类型字段和关系的模式层；按模式存储结构化 Markdown 条目的维基层；以及将每个条目链接回原始代码、配置或文档以供验证的原始来源层。该结构使智能体能够在模块、特征、流水线和实现边界层面检索知识，而非依赖粗粒度关键词搜索。

System KB 通过摄取-查询-检查生命周期维护。摄取期间，多智能体提取器将 DSL 定义、C++ 代码、配置文件和工程文档等异构来源转换为标准化维基条目。查询期间，模式感知检索向下游智能体提供候选项特定的上下文，尤其适用于依赖变更实现位置或所需特征是否可用的想法。检查期间，由代码差异触发的增量更新及周期性检查器使知识库与持续演化的系统保持一致。

因此，对于可行性敏感的候选项，System KB 在 Brainstorm Agent 中权重更高。它有助于拒绝违反流水线边界、需要不可用特征、依赖不受支持的 DSL 行为或无法交付到正确代码范围的想法。通过以已验证系统知识为依据，System KB 将架构和实现约束转化为候选项评分的显式证据。

4.3.3. Data Analysis

Data Analysis 为有效性依赖观测数据模式而非仅凭直觉的候选项提供实证。必要时，它可访问历史分析报告、指标定义、SQL 方案、离线统计和当前 SQL 查询结果。这使其特别适用于依赖用户行为、流量分布、特征覆盖、样本偏差或指标分解等假设的想法。

对于每个候选项，Data Analysis 帮助验证底层实证前提是否确实成立。例如，在提出针对长尾用户、冷启动物品或特定失败分群的想法前，智能体可以检查分群是否足够大、指标下降是否具有统计意义，以及提议方向是否与观测数据模式一致。这可以防止 Brainstorm Agent 过拟合轶事观察，或仅依据表层指标变化提出改动。

当候选项依赖指标诊断、分群行为、分布漂移或其他数据驱动主张时，Data Analysis 会获得更高权重。它就问题是否真实、目标分群是否值得优化，以及所提机制是否符合观测数据提供证据。通过以实证分析为依据，它将头脑风暴从直觉驱动的提案生成转化为数据感知的假设形成。

4.3.4. Model Research

Model Research 为价值依赖最新论文或既有学术发现的候选项提供外部研究证据。其职责是将研究论文转化为可执行的提案知识，而不是将其视作普通文本摘要。这很重要，因为诸如新门控机制

或序列模块之类的论文级主张，可能在受控基准上有效，却在生产推荐系统的约束下不可行或无效。

为支持这一转换，**Model Research** 为摄取的论文维护结构化事实库。每篇论文被分解为类型化主张、架构组件和论文间关系。主张按问题、假设、方法、发现或局限等角色，以及独立消融、系统级评估、报告结果或作者主张等证据强度进行标注。架构条目描述推荐特定组件，包括骨干网络、特征交互算子、目标和预测头。论文间边记录论文彼此间的扩展、矛盾、并行或应用关系。该结构使智能体能在可复用机制层面推理论文，而不依赖粗粒度论文摘要。

生产基线及其特征契约和训练约束使用同一模式表示，使论文衍生想法能与真实系统对齐，而非抽象评估。特征契约枚举每个特征槽的名称、维度和嵌入表标识；保留该契约的提案可直接从生产检查点初始化，仅训练新增的稠密参数。训练约束是硬性的：流式在线增量训练、无 **epoch**，并禁止骨干冻结和早停等技术。这使 **Model Research** 能检查论文衍生想法是否保留所需特征槽、适配现有模型架构并遵守生产训练规则；违反约束的想法在传递给下游前即被拒绝。因此，搜索空间从源头就受实际生产架构和训练范式约束，而不是事后仅以生产指标评估——这正是它与搜索空间和奖励信号均为合成的学术自动研究系统之间的核心工业差异。

当候选项依赖外部新颖性或近期学术证据时，**Model Research** 权重更高。它有助于识别哪些论文主张可能迁移、哪些模块值得复现或消融，以及哪些跨论文组合可能有潜力。通过将论文知识与生产约束对齐，**Model Research** 将近期研究转化为可行实验提案，而非未经验证的灵感。这些提案不限于单篇论文复现：同一套对齐后的知识会驱动系统化探索循环——复现、模块消融和跨论文组合——其执行和自我演化见第 5.2 节和第 7.2 节。

4.4. 验证与实现交付

候选方案生成后，任何想法在成为编码任务之前都必须经过准入步骤。验证器会检查主要目标的一致性、业务语义正确性、用户约束、模型分数语义、实现可行性、历史重合度、A/B 参数可行性以及成熟度一致性。该审查同时产出轮次级和候选级决策。只有通过验证且准备就绪的候选方案才能进入人工审批关口；需要优先探查的想法仍作为显式的证据收集任务，而待办想法则继续留在执行队列之外。

人工审查被视为一个聚焦的准入关口，而不是修复每个薄弱候选方案的机制。需要人工审查时，审查者从机器筛选出的方向中选择批准、修订、推迟或拒绝。对于每个获批想法，实现移交会创建且仅创建一条正式实验记录、一份来源清单以及一份交付 **Developing Agent** 的移交计划。移交内容包括目标行为、所需信号、预期指标路径、护栏和已知实现边界，但刻意不在头脑风暴阶段编写生产代码。

这种职责分离使该模块能够在 **AgentX** 内部进行组合。**Brainstorm Agent** 负责减少歧义、进行证据加权和提案准入。**Developing Agent** 负责代码实现和仓库级验证。通过强制落实这一边界，**AgentX** 既能防止模糊想法渗入实现，又能让系统探索明显局部改动之外的空间。

5. 开发智能体

Developing Agent 是将获准提案转化为可验证代码制品的模块，它沿着两条并行路径运行，映射了推荐系统在实践中迭代的方式。

在**线上策略**路径中，制品是针对特征级或策略级提案的生产代码变更，目标是在不出现隐性可靠性故障的前提下安全服务线上流量。在**离线模型**路径中，制品是针对模型架构提案的训练实验，目标是产出足够可信的结论，使其能够沉淀到长期维护的模型探索发现知识库中。只有同时满足四个条件，结论才被视为可信：实现经验证与策略声明的因果机制及预期可观测量一致；相互隔离的专家智能体小组对策略达成绝对多数共识；指标以确定性方式从原始训练日志中提取，而非由 LLM 解读；任何报告的 AUC 增益均有经过验证的因果链归因支持，未归因的增益应作为制动信号，而不能仅凭数值改善予以记录。

尽管交付物不同，两条路径都面临同一根本风险：有前景的想法在实现阶段退化为隐性故障。在线上，目标代码仓库庞大，内部约定仅有部分文档，而且许多错误无法被编译期检查发现：错误的特征名虽能通过编译，却读取无意义的数据库；缺失工厂注册会在无通知的情况下禁用策略；未加保护且默认开启的变更可能在审查前影响线上流量。在线下，风险以类似方式表现：幻觉指标和未经验证的因果机制同样会悄无声息地污染知识库，且没有明确的失败信号。

因此，Developing Agent 并非被设计为通用代码生成器，而是一个面向验证的实现系统，其规范统一适用于两条路径。本节其余部分先在第 5.1 节详述线上策略路径，再在第 5.2 节将同一组原则适配到离线模型路径。

5.1. 线上策略开发

5.1.1. 生产代码可靠性问题

AgentX 中的编码契约比生成语法有效的补丁更严格。智能体必须保持提案意图，留在允许的代码边界内，仅使用已验证的仓库原语，通过本地和集成检查，并留下可安全上线、可供评审的变更。核心困难在于这些要求相互作用。补丁可能在逻辑上符合提案，却使用了幻觉属性；也可能能够编译，却遗漏所需的流水线注册；还可能正确实现策略，却在没有默认关闭保护的情况下将其激活。

因此，主要失败模式并非通用编程错误，而是仓库特定的可靠性失败。属性幻觉发生于智能体在用户特征模式、上下文特征模式或物品特征模式中虚构字段。DSL 误用发生于猜测排序 DSL 算子名称或参数契约。Harness 模式违规发生于变更被置于错误队列、注册不完整或绕过必要安全模式。最后，每一次额外的正确性修复循环和人工干预都会降低自动化可靠性，因为 AgentX 的目标是以最少人工救援闭环从想法到上线的循环。

5.1.2. 仓库依据的代码生成

Developing Agent 通过将实现建立在两类代码仓库知识之上来应对这些风险。第一类是项目特定知识库，记录变更模式、注册约定、特征开关规则以及已获准补丁的示例。第二类是案例工具箱：一

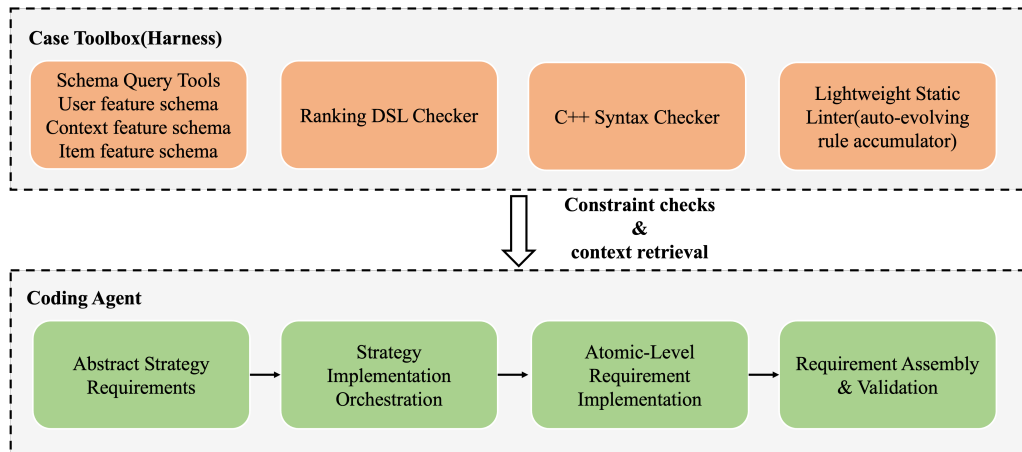


Figure 4 | 在线策略赛道 Developing Agent 的架构。中央流水线通过四个顺序阶段，将策略规范转化为通过验证的代码提交：抽象需求、编排实现计划、执行原子子需求，以及组装并验证。案例工具箱（右侧）向流水线注入约束检查和上下文检索能力，其中包括面向用户侧、上下文侧和物品侧特征结构体的模式检索工具、排序 DSL 检查器、C++ 语法检查器，以及一个可自动积累项目特定规则的轻量级静态代码检查器。

组确定性工具和检查器，迫使智能体在使用事实前先进行验证。二者共同将编码任务从开放式生成转化为一系列有事实依据的决策。

最重要的事实依据规则是：特征属性必须先查询后使用。面向用户侧、上下文侧和物品侧特征结构体的 `schema` 查询工具会返回各领域可用的字段。智能体在读取属性前必须调用相关工具，从而使字段名成为经过验证的事实，而非语言模型的猜测。对于排序 DSL 调用，基于编译器的检查器会验证算子名称、参数类型和调用契约。对于 C++ 惯用法和项目特定语法糖，快速轻量的静态检查器会在补丁进入更重型的构建流水线前捕获违规。

5.1.3. 面向验证的实现循环

每项编码任务都遵循一个分阶段循环。智能体首先将获批提案抽象为实现计划：哪些文件可以改动、需要哪些信号、逻辑归属哪个流水线阶段、由哪个特性开关保护该行为，以及提交前必须通过哪些检查。随后，它实现原子子需求、组装补丁，并运行确定性验证。失败结果会被反馈为有针对性的修复指令，而不是宽泛的重新生成提示。

其中两层验证尤为重要。准确性循环将实现与计划进行比较，并把额外的修复迭代次数计为质量成本。理想轨迹是首次实现即已符合指定策略。随后，*dryrun* 流水线会对分支进行编译和集成检查。干净的轨迹只需一次即可通过 *Dryrun*；反复出现 *Dryrun* 失败，表明智能体正在将远程基础设施当作调试工具，而非在本地生成严谨规范的补丁。下文的质量评分同时保留这两层因素。

5.1.4. 质量评分

编码质量被衡量为加权可靠性分数，而非单一的通过/失败结果。令 $\mathcal{N} = \{1, \dots, 8\}$ 表示八个观测失败维度， $s_i \in [0, 1]$ 表示维度 i 的归一化成功分数。总体编码分数计算如下：

$$Q_{\text{code}} = \sum_{i \in \mathcal{N}} \lambda_i s_i, \quad \sum_{i \in \mathcal{N}} \lambda_i = 1. \quad (5)$$

在当前实现中，各权重具体设定为：

$$Q_{\text{code}} = 0.06s_1 + 0.12s_2 + 0.22s_3 + 0.08s_4 + 0.06s_5 + 0.18s_6 + 0.18s_7 + 0.10s_8. \quad (6)$$

Table 1 | 八维编码质量评分因素。

因素	含义	严重度	权重
N_1	C++ 语法糖违规	B	6%
N_2	Harness 模式违规	A	12%
N_3	属性幻觉	S	22%
N_4	排序 DSL 检查修正	A	8%
N_5	C++ 语法检查修正	B	6%
N_6	正确性循环迭代	S	18%
N_7	人工操作干预	S	18%
N_8	Dryrun 流水线通过次数	A	10%

各维度均归一化为越高越好。对于基于计数的失败， s_i 会随修正或违规次数增加而下降。人工干预被视为硬二元门控：无人工干预时为 $s_7 = 1$ ，否则为 $s_7 = 0$ 。Dryrun 相对于一次通过的理想状态评分，即 $s_8 = \max(0, 1 - (N_8 - 1)/2)$ 。属性幻觉、正确性循环开销和人工干预这三个 S 级维度合计占总权重的 58%，因为它们对自主生产可靠性的威胁最直接。

5.1.5. 失败模式

这些评分维度对应具体的实现失败。缺少模式依据时，智能体可能读取一个看似合理但实际不存在的物品属性；这类问题可能通过表面化的代码审查，却会破坏线上行为。例如，在强制使用模式工具之前，智能体可能在物品属性结构体中虚构诸如 `author_live_cpm_boost` 的字段。代码在上下文中看起来合乎情理，但正确字段必须通过物品特征模式工具解析；否则，实现将读取无效或非预期的属性。缺少 DSL 检查时，智能体可能虚构算子，或以不受支持的形式传递参数。一种反复出现的模式是创建名称听起来合理、但并未注册的排序算子，尽管已注册的算子契约要求使用不同的评分器接口。DSL 检查器将这种原本会在后期编译或审查阶段暴露的失败，转化为有针对性的本地修复。缺少注册模式知识时，智能体可能实现了评分器，却未能将其接入服务流水线。在这种情况下，核心评分文件可能正确，但缺少工厂或队列注册，导致策略在线上悄然失效。这正是文件覆盖检查和测试框架模式检查需要与普通语法检查相互独立的原因。缺少特性开关规范时，智能体可能默认启

用某项策略，而不是将其隐藏在默认关闭的开关之后。预期模式是：除非启用专用特性开关，否则基础分数保持不变；无条件的分数乘法会被视为违反默认安全原则。

Developing Agent 的目的是在补丁进入生产审查之前，让这些失败变得可观测。因此，它对 AgentX 的贡献并不只是实现吞吐量。它提供仓库级可靠性层，使 Brainstorm Agent 和 Model Agent 的提案能够转化为安全、可检查的代码改动。

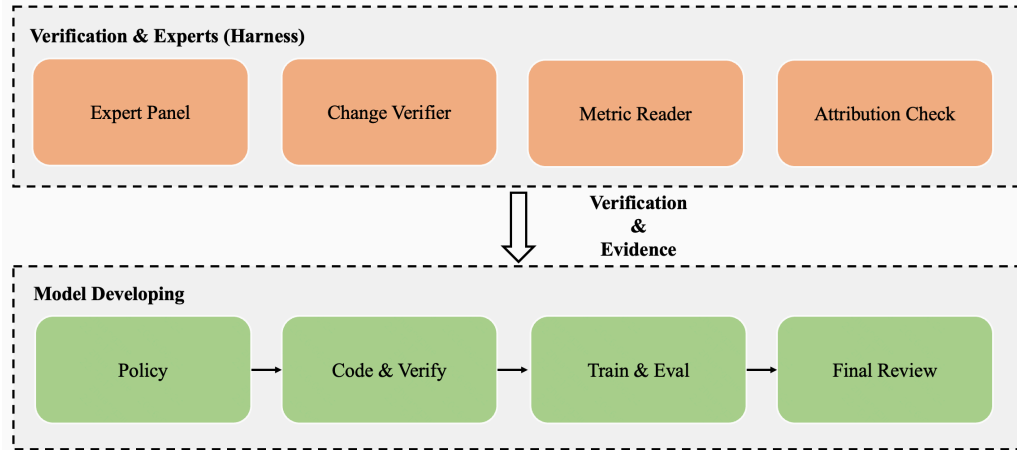


Figure 5 | 离线模型轨道 Coding Agent 的架构。中央流水线（下方）通过四个顺序阶段，将模型架构提案转化为可信训练结论：声明策略、实现并验证代码变更、运行训练与评估，以及结合因果链归因评审结果。验证与专家 harness（上方）通过隔离的专家组投票达成共识、确认代码与策略一致的变更验证器、从训练日志提取 AUC 的指标读取器，以及判定因果链各环节是否成立的归因检查来注入检查和证据。

5.2. 模型开发

模型架构提案的开发方式不同于在线生产变更。其目标不是生成安全、可部署的补丁，而是完成可信的离线训练实验：智能体必须实现所提架构，在生产平台上运行真实的训练与评估作业，并返回可信结论。整个链路很长——解读提案、编辑模型代码、验证变更、向共享集群提交训练作业、等待结果、解析训练日志并综合裁决——每一步都可能失败：LLM 可能误解方向、编写存在细微错误的代码、幻觉出指标，或将相关性与机制混为一谈；训练平台还会带来 GPU OOM、瞬时基础设施故障和日志停滞。对于持续数日的一批提案，任一问题都可能污染反馈到探索记忆中的结论。执行智能体围绕一个原则构建：LLM 只允许在判断上出错；所有客观事实均由确定性代码产生。

5.2.1. 单轮流水线

研究循环每一轮遵循如下结构：

$$\text{policy} \rightarrow \underbrace{(\text{code} \leftrightarrow \text{verify})}_{\leq 3 \text{ rewrites}} \parallel \underbrace{\text{experts} \times N}_{\text{concurrent}} \rightarrow \text{exec} \rightarrow \text{final_review}.$$

本轮从 **policy** 开始，它不仅声明要改变什么，还声明所主张的因果机制和一组预期可观测量——每个名称都精确定义，使代码智能体可将其接入 `tf.print` 或 `tf.summary`，并描述健康 vs. 异常行为（例如，“第 1000 步后门激活 > 0.5 表示门处于活跃状态； ≈ 0 表示坍塌”）。这迫使本轮预先承诺如何被证伪，而非事后解释结果。**Code** 在指定文件边界内实现策略。随后 **Verify** 检查两点：生成的 `git diff` 在语义上是否符合策略方向，以及每个已声明可观测量名称是否出现在 `diff` 中。任一检查失败，代码智能体都会重写，最多三次；预算耗尽则干净地终止本轮，而不会继续执行未经验证的代码。与此循环并发，**expert agents** 独立评估策略，每个专家只接收策略文本和自己的私有知识库，在物理上与历史和其他专家意见隔离。专家共识由 **Python** 计票（ $\geq \lceil 2N/3 \rceil$ 表示绝对多数）计算，绝不由 LLM 决定。**Exec** 是不含 LLM 的纯 **Python** 状态机，在提交、轮询、评估和指标提取状态间转换，通过正则表达式从原始训练日志中提取 AUC。此处刻意排除 LLM：指标提取是存在正确答案的模式匹配问题，而幻觉 AUC 数值写入知识库的代价很高。

5.2.2. 可证伪归因

Final review 将指标、专家意见和训练日志综合为裁决。关键是，它还会将训练日志观测与策略中声明的预期可观测量比较，逐一判定所声明因果链的每个环节为 `verified`、`broken` 或 `unclear`。随后 **Python** 确定性地归并该列表：所有环节均为 `verified` 时，归因状态为 `CLEAR`；任何 `broken` 或 `unclear` 环节都会得到 `UNCLEAR`。裁决与归因状态是两个独立字段，这种分离至关重要：若某轮 AUC 提升但归因不明确，它就是一个制动信号——系统不会传播未经归因的增益。**RankMixer** 骨干网络上的实验说明了为何需要这一约束。第 1 轮以正确代码复现论文的乘法门 $x \cdot \tanh(V_o x)$ ，AUC 小幅提升 $+0.0003$ 。若无因果链强制机制，这会被记录为成功。但门激活的声明可观测量在整个训练期间接近零——分析确认， V_o 的 **Glorot** 初始化产生了 $V_o x \approx 0$ ，进而导致 $\tanh(\cdot) \approx 0$ ，使门输出归零并完全阻断梯度。归因状态：`UNCLEAR`。该增益未被记录。第 2 轮应用单行残差修复 $x \cdot (1 + \tanh(V_o x))$ ——当 $V_o \approx 0$ 时退化为恒等映射，从而恢复梯度流——并返回 $\Delta AUC = +0.0022$ ，且所有因果链环节均为 `verified`。

5.2.3. 平台故障下的稳健执行

在生产训练环境中，许多失败与提案本身无关：GPU 内存溢出、集群作业停滞、基础设施连接超时。若统一处理这些失败——总是重试或从不重试——系统要么在根本有缺陷的配置上浪费算力，要么被瞬时噪声终止。因此，执行智能体会在决定响应方式前对每个失败分类。纯函数分类器读取日志头（8 MB）、日志尾（256 KB）以及从中间采样的最多 64 条 **FATAL** 行，并将失败映射到特定原因码。确定性错误——NaN 梯度、特征表冲突、缺少评估检查点——会立即放弃，因为重试无济于事。瞬时故障——通过日志尾字节数和哈希指纹检测到的日志停滞、基础设施中止——重试一次。LLM 网关故障则切换至下一个可用网关。分类器按固定优先级评估原因码；诸如 `ps_aborted` 这类可能掩盖底层确定性错误的基础设施症状始终最后评估，防止将模式错误误判为可恢复的瞬时故障。独立运行的看门狗守护进程对批次中的所有活跃运行应用相同策略，无需人工干预即可自愈停滞或失败的运行。每个被放弃的运行都会明确记录原因码（例如 `given_up:nan_train`），使根因一目了然。这种执行层稳健性在正常运行时不可见，却是系统连续多日并行运行数十个实验的必要条件。

6. 评估智能体

Evaluation Agent 闭合了 AgentX 的生产闭环：它判断由 **Developing Agent** 实现的代码改动应当保留、回滚，还是作为负面经验反馈给下一轮迭代。因此，它的职责不只是报告 A/B 数值，而是将嘈杂、延迟且部分可观测的线上流量转化为系统其他部分可据以行动的可信奖励信号。为此，**Evaluation Agent** 被设计为端到端流水线，而非单一判断模块，从而将真实用户反馈稳定地转化为 AgentX 的权威奖励信号以及塑造未来迭代的可复用约束。

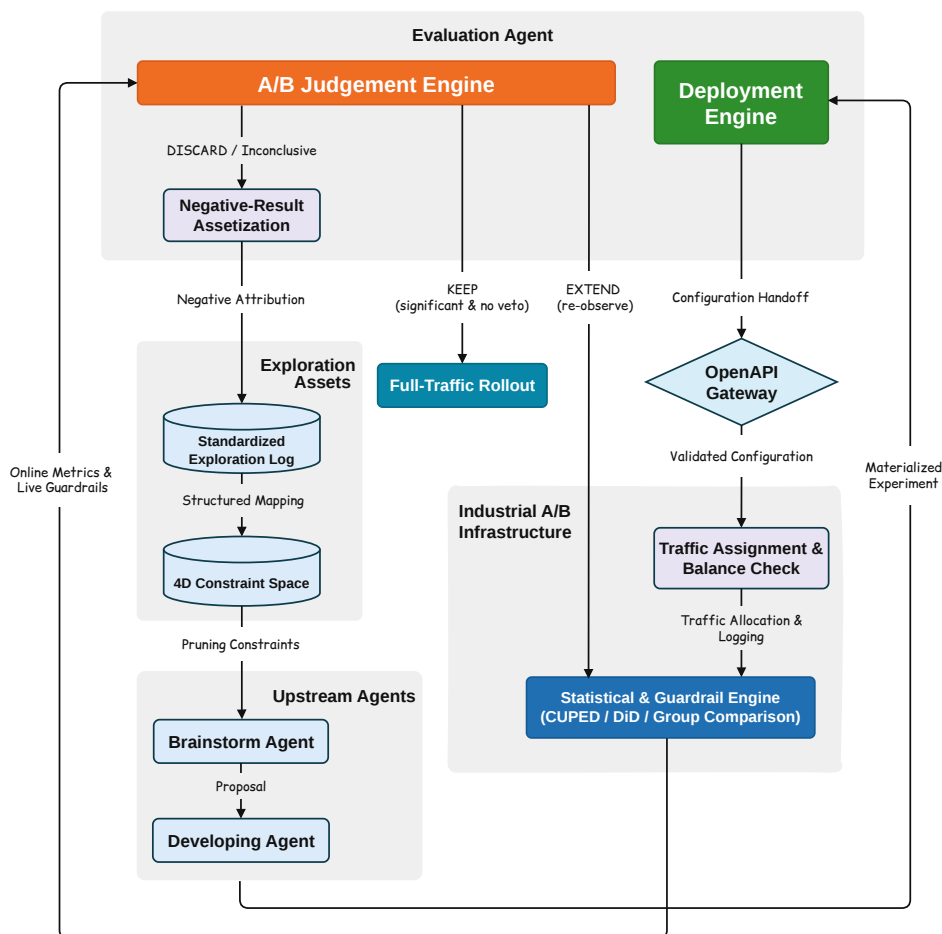


Figure 6 | **Evaluation Agent** 的架构。它串联由 **OpenAPI** 中介的安全部署、线上 A/B 执行和护栏否决判定，将生产反馈转化为后续 **AgentX** 迭代的奖励信号。

6.1. 真实世界奖励问题

Brainstorm Agent 提出实验、**Developing Agent** 将其实例化之后，系统仍然不知道该改动是否值得保留。在推荐系统中，离线代理指标和自我反思并不足够：某项策略可能提高内部得分，却损害长期用户体验；也可能在小规模切片中看似有效，却破坏某项护栏指标。因此，**AgentX** 将线上 A/B 反馈视为系统迭代的权威奖励信号。

这又带来了第二个可靠性问题。智能体必须安全地启动实验，在不污染其他实验的前提下分配流量，解读嘈杂的线上指标，应用严格的发布标准，并将负面结果转化为可复用记忆。Evaluation Agent 正是围绕这一完整链条设计的。它不仅是报告模块，更是将真实用户反馈转化为决策和未来约束的机制。

6.2. 安全部署与流量分配

实验必须先安全进入流量，才能产生奖励。工业推荐系统通常会通过多个业务域、层级、world、桶区间和分流因子路由请求。若自动生成的策略上线时不遵循这一拓扑，可能导致参数冲突、跨实验污染或用户分配不一致。因此，Evaluation Agent 从部署寻址开始。

部署层将每个实验映射到正确的业务域和 world，选择适当的分流因子，并分配互斥流量桶。对于账号绑定实验，可依据用户 ID 路由；对于设备侧体验变更，可使用设备 ID；对于混合人群，UID 优先策略可保持登录用户的一致性，并对匿名流量回退到设备身份。当桶分配无法唯一确定时，智能体会执行实验前平衡检查，并选择使基线失配最小的流量组。

发布前会应用安全控制。参数变更必须通过工程白名单，防止智能体修改未经授权的调度或服务控制项。配置发布遵循金丝雀路径：API 提交变更后，系统先观察一个最短灰度发布窗口，再扩大到全量流量。若监控检测到不稳定，发布会在实验演变为大规模线上风险前停止。

6.3. 带护栏否决的 A/B 判定

流量上线后，智能体必须判断观测到的效果是否可信。线上数据嘈杂、延迟，有时还会部分缺失。因此，Evaluation Agent 将指标提取与决策逻辑分离。当存在实验前历史数据时，它采用 CUPED 等方差缩减方法。当数据环境不满足这些假设时，则退回到双重差分或直接组间比较等更稳健的方法。如果上游数据缺失，智能体会缩短或平移观测窗口，而不是将不完整查询当作结论。

决策策略被有意设计得较为保守，但其护栏设计遵循三项原则，以控制假阴性。

第一，**护栏按业务范围设定，而非全局通用**。每个业务领域（如消费、直播、电商、广告）都有自身的核心指标和否决阈值，并针对其具体优化目标进行校准。某一领域的实验不受系统全部护栏并集的约束；它主要对本领域的护栏指标以及少量跨领域稳定性指标负责。

第二，**护栏采用复合经济交换指标，而非单指标否决**。系统不会因任一指标越过硬阈值便阻断实验，而是计算聚合的生命周期价值（LT）交换分数，将多个业务目标加权为统一摘要。单项护栏指标负向变化不会自动触发否决；智能体会考察综合表现，从而在仍能捕获策略真正损害生态的情况下，大幅减少噪声导致的假阴性。

第三，**阈值被设计为关注信号，而非绝对阻断条件**。护栏阈值的主要作用是标记风险供人工审查，而非机械地丢弃实验。触发护栏时，策略会升级审查，而不会自动丢弃。实践中，收益较高但引发中度护栏恶化的策略仍可进入例外审查路径，前提是收益足以补偿外部性。这种分层设计——严重恶化时硬阻断、中度信号时升级审查、观测指标仅监控——既维持安全性，也避免系统过度保守。

这些原则共同确保，护栏否决能够捕获真正有害的策略，而不会系统性压制有益实验。只有当主要目标同时超过最小效果阈值并达到统计显著性，且护栏评估确认不存在不可接受的跨领域损害时，候选方案才有资格被判定为 KEEP。

因此，分析阶段的输出并非单个数值。它是结构化裁决：KEEP、EXTEND 或 DISCARD，并附带主要效果、护栏状态、统计方法、观测窗口和注意事项。这一结构化裁决就是 AgentX 闭环后续阶段使用的奖励记录。

6.4. 负面结果资产化

大多数生产实验最终不会成为成功的发布项。对 AgentX 而言，这并非无用功。只要系统记录策略失败的原因，并阻止后续智能体重复同一路径，负向结果就具有价值。因此，Evaluation Agent 会将 DISCARD 和无定论结果转化为探索资产。

每个失败实验都会记录根因：显著性不足、护栏恶化、流量失配、实现侧注意事项或业务情境不匹配。记录还会按流水线阶段、业务目标、受影响的用户或内容分群以及策略杠杆建立索引。在下一轮头脑风暴或分析前，AgentX 可检索这些资产，避免重新探索已经失败的方向。例如，若此前针对低频用户的提权策略在粗排阶段损害了留存护栏，同一坐标组合就会成为后续提案生成中的高风险区域。

这一资产化步骤完成了闭环。线上 A/B 结果决定哪些变更得以保留，而负向结果则决定后续哪些分支应被剪枝或更谨慎地探测。因此，Evaluation Agent 同时提供奖励信号和失败记忆，使 AgentX 的自迭代以真实生产反馈为依据，而非依赖模型自我评估。

7. AgentX 框架演化

生产实验分析只覆盖 AgentX 反馈闭环的一侧。Experiment Analysis Agent 虽能诊断在线结果、记录失败方向并将 A/B 反馈转化为可复用记忆，却仍留下关键缺口：它无法解释上游智能体为何未能捕获正确约束、遗漏业务因果链、产生不完整交付，或生成违反仓库惯例的代码。为弥合该缺口，AgentX 在执行轨迹之上引入第二层优化。其目标不是推荐策略本身，而是控制各子智能体如何推理、提问、验证证据、交付工作并从失败中恢复的 harness。

我们以严格受限的含义使用 *harness* 一词。完整运行 harness 包括顶层编排、记忆、工具接口、智能体间协议和子智能体指令，但当前生产安全版本一次只更新一个子智能体级 harness 规格。在一次演化运行中，基础模型、工具接口、顶层编排和其他子智能体保持固定；仅编辑目标子智能体的指令、验证规则、输出契约和工具使用纪律。该设计使更新完全可审查，并使新旧回放具有意义：任何分数变化都可归因于局部 harness 编辑，而非大规模系统重写。针对这一受限设定，我们提出 **Semantic-Gradient-based Prompt Optimization (SGPO)**，这是一种离线 harness 演化方法，可将累积执行轨迹转化为局部子智能体提示词更新，并仅通过配对回放接纳更新。简言之，SGPO 将对轨迹失败的自然语言诊断视为语义梯度，在 AgentX 其余部分保持固定时修订单个子智能体规格。

7.1. 基于语义梯度的提示词优化

7.1.1. 基于语义梯度的提示词优化 I

SGPO-I 使用会话轨迹作为证据来源。令 $h_{t,i}$ 表示目标子智能体 i 在演进轮次 t 时的当前 harness 规范。每轮中，SGPO 从累积线上轨迹池中采样一小批与子智能体 i 相关的轨迹。评估器不会逐字读取完整会话，而是从初始用户查询和后续用户输入中提取紧凑的评分准则，因为这些字段通常包含显式任务约束以及交互过程中显现的隐式约束。

第一步是计算损失，产生用于改进 harness 的语义梯度。给定当前 harness、采样轨迹证据 ($\mathcal{T}$) 和提取的评分准则 ($\mathcal{R}$)，评估智能体 (E_{agent}) 先撰写自然语言损失报告，再将其压缩为语义梯度 $g_{t,i}$ ：

$$\ell_{t,i}, g_{t,i} = E_{\text{agent}}(h_{t,i}; \mathcal{T}, \mathcal{R}). \quad (7)$$

语义梯度 $g_{t,i}$ 并非数值导数。它是对约束缺失、步骤顺序薄弱、证据要求欠明确或下游契约不完整的结构化诊断。

第二步是语义梯度更新。优化智能体 (R_{agent}) 将该梯度转化为局部框架编辑，并生成候选修订框架：

$$h'_{t,i} = R_{\text{agent}}(h_{t,i}, g_{t,i}). \quad (8)$$

细化仅限于目标子智能体的指令、验证规则、输出契约或工具使用纪律；它不会重写完整 AgentX harness。

生成 $h'_{t,i}$ 后，SGPO 立即在同一组回放任务上评估旧、新 harness。回放任务集从同一轨迹池生成：LLM 将用户查询和后续用户输入重写为独立用户任务。每个回放任务保留业务领域、目标、护栏、允许的变更类型、预期产物和已知约束，同时消除对早先对话上下文的依赖。随后，AgentX 在相同任务下分别使用旧 harness $h_{t,i}$ 和候选 harness $h'_{t,i}$ 重新运行，评估器用同一评分量表族对二者输出评分：

$$\begin{aligned} \Delta J_i &= \text{ReplayScore}(h'_{t,i}) - \text{ReplayScore}(h_{t,i}), \\ h_{t+1,i} &:= h'_{t,i}, \quad \text{若 } \Delta J_i > \epsilon \wedge \text{Safe}(\Delta h_i). \end{aligned} \quad (9)$$

其中， ΔJ_i 是成对回放改进量， ϵ 是准入阈值。若 $\Delta J_i > \epsilon$ ，且安全检查确保 schema 兼容性、工具边界和人工审查约束得到保留，则接受候选更新，并在后续迭代中以更新状态 $h'_{t,i}$ 作为 $h_{t+1,i}$ 。否则，不更新目标子智能体，并保留被拒补丁、评估器得分和失败说明，作为后续轮次的改进经验。

该优化流水线通过三阶段闭环（如图 7 所示）保持清晰的可检查性和严格的生产安全性。

1. **采样与评分准则构建（步骤 1）**：从累积轨迹池抽取执行轨迹形成采样轨迹。同时，利用 *Exp User Tasks* 数据库中的历史评估任务构建显式评分准则（如 `user_query` 和 `user_content`）。
2. **语义梯度估计与改进（步骤 2）**：为计算语言层面的“损失”，*Harness Evaluate Agent* 对照当前目标 *Harness* (h) 分析采样轨迹中的失败。诊断产生结构化的**语义梯度**，明确映射需求缺失、步骤顺序错误、工具使用违规或输出契约破坏等执行缺陷。作为提示空间优化器，*Harness Refine Agent* 利用该梯度提出更新后的**候选 Harness** (h')。

3. **成对回放与经验吸收（步骤 3）**：为严格把控生产部署，*Harness Exp Agent* 执行**成对回放**评估；具体而言，它在相同用户任务上同时运行当前 harness $h_{t,i}$ 和候选 harness $h'_{t,i}$ 。*Harness Exp Agent* 评估性能差值 ($\Delta score$)。若回放产生正向证据 ($\Delta score > 0$)，候选 harness 被正式**接受**并投入生产；反之则触发 **No-op**，并将失败模式归档到 *Refine Experience* 数据库，以指导后续优化迭代。

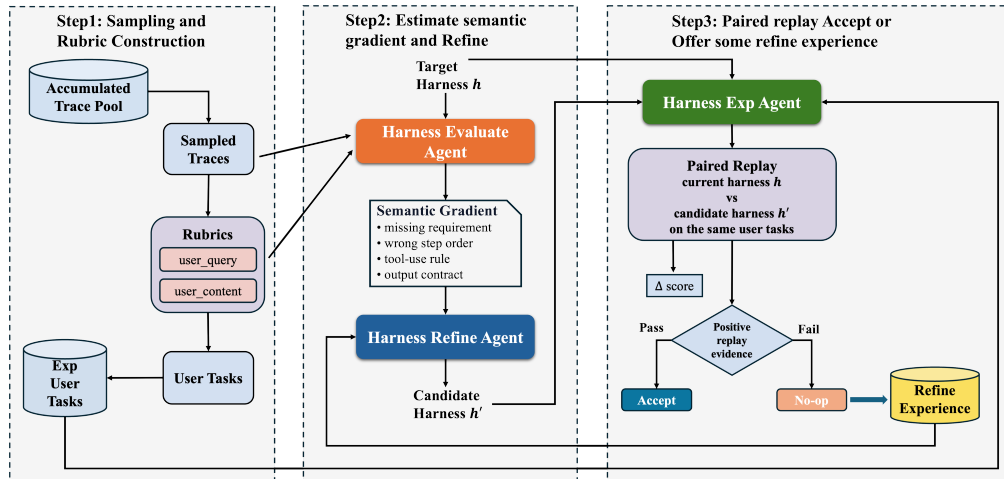


Figure 7 | SGPO-I 基于轨迹的框架演化。累积轨迹被采样为评分准则和回放任务；评估器反馈被转化为语义梯度，进一步优化为候选框架，并且只有通过配对回放后才会被接纳。

在一个生产用例中，SGPO-I 被应用于想法生成阶段，该阶段的失败通常表现为目标模糊、缺少证据依据、方向重复，或产物无法被下游验证和编码智能体直接使用。表 2 总结了 Brainstorm 子智能体的一次五轮演化。归一化回放分数从 75.15% 提升到 98.00%。最重要的已接纳编辑并非泛化的提示词润色，而是具体的契约变更，例如在生成想法前明确任务契约，并要求每个候选项呈现业务因果链。

Table 2 | Brainstorm 子智能体的 SGPO-I 演化轨迹。每轮采样轨迹、提出局部 harness 编辑，并仅在配对回放后接纳该编辑。

轮次	编辑重点	对框架的影响	平均得分	归一化
1	任务契约规范化	生成候选方案前，重述业务领域、目标指标、范围、人工审查要求、硬约束和缺失信息。	3.76	75.15%
2	证据依据	要求提供显式证据索引，并标记不可用证据，而不是默默依赖直觉。	3.89	77.82%
3	候选质量模式	围绕问题来源、业务因果链、策略公式、实现点、dry-run 检查和风险，对每个想法进行标准化。	4.53	90.67%
4	新颖性与拒绝	区分主题相同但切入点新颖的想法与重复优化点，并记录被拒方向及原因。	4.87	97.33%
5	可评估移交	为每个想法生成候选概览、优先级顺序、验证 / 编码字段、不受支持项和评估标准。	4.90	98.00%

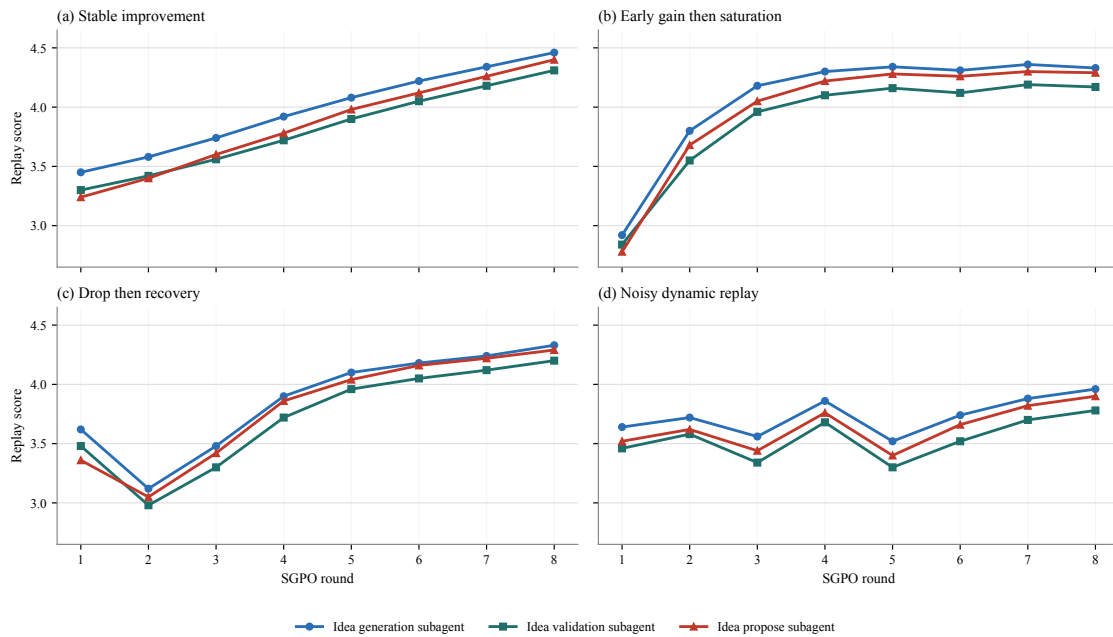


Figure 8 | Brainstorm 工作流中具有代表性的 SGPO-I 演进模式。每个面板展示固定设置下的八轮 SGPO：轨迹批大小为 5、8 个评分准则项、10 个回放任务。三条曲线分别对应想法生成子智能体、想法验证子智能体和想法提案子智能体。由于评分准则和回放任务是动态重新生成的，harness 演进可能呈现稳定增益、早期饱和、暂时回退或噪声恢复，而非严格单调的趋势。

我们进一步在固定运行设置下考察头脑风暴工作流中的 SGPO-I。图 8 总结了框架演化历史中观察到的四种代表性演化模式。每个面板包含八轮 SGPO，并跟踪想法生成子智能体、想法验证子智能体和想法提案子智能体。曲线表明，SGPO-I 不一定单调：根据采样轨迹和回放任务的不同，某一轮可能带来稳步提升、早期饱和、暂时回退或在噪声中恢复。这些模式说明，SGPO-I 应被理解为

门控搜索过程，而非单调优化器。暂时的分数下降符合预期，因为每轮都会采样新轨迹，并重新生成评分准则和回放任务。关键并非每次候选更新都能改善每个回放集，而是配对回放能在回退被接纳入生产框架之前将其暴露出来。

7.1.2. 基于语义梯度的提示词优化 II

作为 SGPO-I 的扩展，SGPO-II 将证据来源从对话轨迹改为由历史已合并请求（MR）派生的编码回放案例。之所以需要这一变化，是因为 Developing Agent 的失败具有长时程和代码仓库特异性：生成的补丁可能满足表面需求，却违反所有权边界、特征开关规范、本地辅助函数约定、回滚预期或确定性验证规则。因此，MR 历史充当编码轨迹池。回放前，SGPO-II 会过滤该池，移除噪声、过时、琐碎或批量变更，包括批量格式化、回退、依赖升级、变更少于 10 行的补丁、超过 5,000 行的重写、生成文件占比过高的变更，以及原始 diff 上下文在当前代码库中已不复存在的 MR。保留的样例均为经人工批准且代码仓库上下文仍然有效的代码变更，适合作为编码 harness 演进的证据。

Table 3 | 编码补丁回放的五个评估维度。

Dimension	Weight	Criterion
语义正确性	40%	核心逻辑在语义上等价于真实答案；边界条件正确（ <i>hard gate</i> ：要求分数 ≥ 4 ）
需求覆盖度	25%	满足需求规格中的全部验收标准
文件覆盖度	20%	修改文件集合与已落地补丁一致；无重大遗漏
默认安全性	10%	新行为由默认关闭的特性开关保护；不对现有逻辑作无保护变更
代码风格一致性	5%	命名、结构和注释遵循仓库惯例

SGPO-II 从该清洗后的池中采样回放案例，并将每个保留的 MR 转换为仅包含需求的任务，类似 SGPO-I 中的用户任务。已落地补丁对 Developing Agent 隐藏，仅在智能体生成实现后供评估器作为参考。编码开始前以 5 个一批的方式合成案例，每个案例都在从 MR 基础提交初始化的干净分支上尝试。评分量表 $\mathcal{R}$ 专门面向编码：语义正确性、需求覆盖度、文件覆盖度、安全性和风格一致性（表 3）。仅当加权总分至少为 4.0，*and Semantic Correctness* 维度至少为 4 时，评估器才判定案例通过。回放失败时，评估器将失败总结为语义梯度，并交给类似 SGPO-I 的 *Harness Refine Agent*。在 SGPO-II 中，细化智能体将梯度转换为 Developing Agent harness 规格的局部更新，具体细化其执行规则、评估器经验、模式记忆和确定性预检查。因此，SGPO-II 保留类似的梯度优化范式，同时用仓库依据的编码证据替代会话轨迹。

在我们的生产用例中，SGPO-II 被应用于一个复杂异步模块，该模块需要多文件协调、异步回调流程管理以及全面的错误处理。表 4 总结了优化前后的快照；图 9 展示了各回放案例中每个维度的改进轨迹。

值得注意的是，迭代流水线并非对所有输入都一致地单调改进。我们在评估集中观察到少量回

Table 4 | SGPO-II 案例研究：复杂异步模块自演化前后的对比。框架从先前失败中积累项目特定约束后，总分由 2.60 提升至 4.90 (+88%)。

维度	失败描述（迭代 1）	之前	之后	Δ
语义正确性	异步回调协调不完整；方法偏离参考实现	2.5	5.0	+2.5
需求覆盖度	缺少关键异步路径；实现支离破碎	2.6	4.9	+2.3
文件覆盖度	多文件修改不完整；缺少协同改动	2.7	4.9	+2.2
默认安全性	多条路径上的错误处理不完整	2.5	5.0	+2.5
代码风格一致性	格式和命名规范不一致	2.6	4.8	+2.2
总体（加权）		2.60	4.90	+2.30

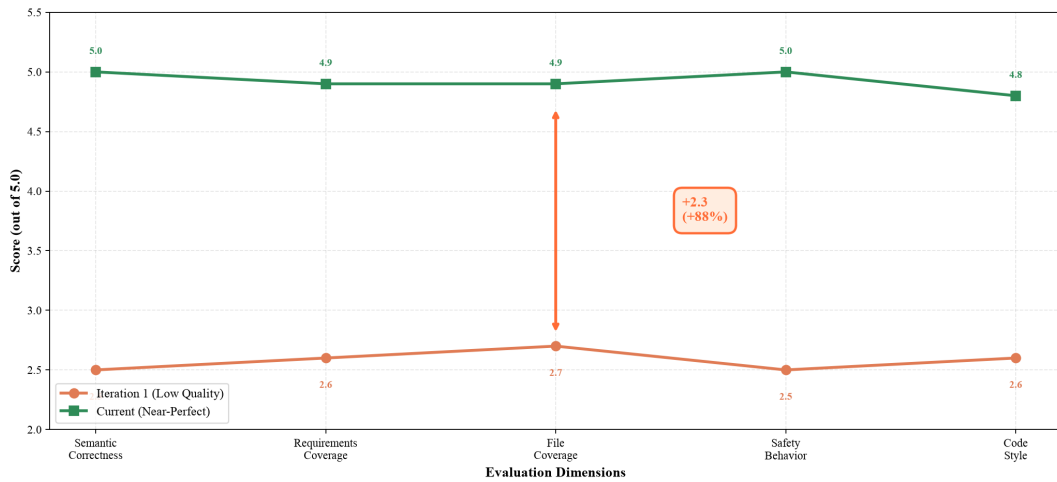


Figure 9 | 案例研究：一个复杂异步模块在 SGPO-II 自进化迭代中的分维度得分轨迹。harness 从 2.60 收敛至 4.90，其中语义正确性和安全性达到最高分，风格一致性的剩余差距最小。

退。在一个匿名示例中，得分从 3.67 降至 1.80 ($\Delta = -1.87$)，最终判定为 **FAIL**。此类回退出现在困难或边界条件案例中，此时 LLM-as-judge 误诊失败模式，并提出与既有约束冲突的 harness 编辑。我们明确保留而非过滤这些回退：它们被存入评估记录，用于刻画方法边界并指导有针对性的后续改进。关键在于，成对回放准入门控 ($\Delta J > \epsilon$) 会阻止退化的制品更新进入已接受的 harness 状态。若某案例在候选 harness 下的得分低于当前 harness，该候选就会被拒绝，因此裁判的误诊只会产生一次 no-op，而不会造成永久退化。这使 SGPO 的可靠性取决于准入门控，而非评估器绝不出错。

7.2. 演进模型研究探索流水线

SGPO 演进智能体如何推理：它编辑一个子智能体的 harness—包括指令、验证规则、输出契约和工具规范—并且仅通过成对回放准入该编辑。第二条演进轴关注系统探索什么，即模型架构方向空间本身。此处 harness 固定，而搜索空间随每次实验判定重塑下一步生成方向而不断改进。给定第 4.3.4

节中已填充的事实存储，问题在于如何不依赖人工筛选有前景方向而进行系统探索；答案是一个三阶段穷举闭环，每一阶段均建立在上一阶段的结论之上。

三阶段探索。 阶段 1 为每篇论文生成一个复现提案，目标是其证据最清晰的发现。这些提案被派发至第 5.2 节的执行流水线，结果汇总为相对于生产基线按平均 ΔAUC 排序的排行榜。排名前 $K=16$ 的论文进入阶段 2。该阶段一方面将搜索空间过滤为确实能迁移到生产环境的方向，另一方面记录每篇前 K 论文的架构块中标为 ablatable 的模块。阶段 2 隔离各模块的贡献。对于前 K 论文中的每个可消融模块，系统枚举两种实现选择—原始选择 (orig_choice) 和 LLM 推断的替代选择—并构建仅激活目标模块、同时将其其他前 K 新增项还原为基线等价项的组合。该隔离有意采用严格设计，因为原论文中的捆绑评估常会掩盖真正驱动增益的组件。若模块在所得 32 个实验中至少 24 个产生正 ΔAUC ，则确认其有效。阶段 3 跨论文组合。常规轮次通过嫁接尚未出现在谱系中且增量最高的已确认模块，扩展每篇前 K 论文的谱系，组合源自不同论文的技术。为防止搜索过早收敛到少量结果簇，每四个常规轮次触发一次挑战者轮次，将高效模块应用于排名 17–32 的论文，以维持探索空间的多样性。

记忆引导的剪枝。 对 1,312 篇论文及其模块进行朴素穷举搜索在计算上不可承受，也会重复访问已被证明无效的方向。系统利用记忆机制避免这一问题，根据不断到来的实验结论持续更新两个层级的剪枝决策。在论文层面，若结果否定某篇论文的核心前提在生产环境中成立，则该论文的所有模块都从阶段 2 和 3 中排除。RankUp 论文提供了具体例子：完整复现运行的最佳结果为 $\Delta AUC=-0.0203$ ，训练日志中的可观测量直接反驳了论文关于有效秩正则化可改善线上训练表征多样性的前提。系统没有继续进行模块消融，而是将其记录为论文级剪枝，并加入反模式条目：在投入模块级消融前，先验证结构性前提能否在生产环境中成立。在模块层面，阶段 2 消融若确认模块不存在独立贡献，就会将其从阶段 3 组合中排除。饱和检测提供全局停止标准：若根据提案中离散化架构字段的 SHA-1 哈希计算的新颖性签名重复率连续两轮超过 80%，闭环便终止，而不再生成结构冗余的实验。阶段门控、记忆引导剪枝和饱和检测共同将盲目枚举转化为围绕有效方向不断收缩的搜索。

经验飞轮。 驱动剪枝的结论本身会经过筛选，而非照单全收。这是第 6.4 节负向结果资产化在离线实验中的对应机制：Evaluation Agent 将被丢弃的线上 A/B 结果转为可复用失败记忆，而探索闭环则将每轮已完成训练—无论成功与否—转化为结构化事件，追加到仅追加事件日志；知识库是该日志之上的视图，可随时从头重建。经验被组织为 anti_patterns（失败模式，带有助于识别未来类似失败的日志与 diff 正则表达式）和 playbook（成功配方，在 $\Delta AUC > 0.001$ 时记录）。并非每个实验结果都足够可靠：单次成功可能来自幸运的超参数交互，单次失败也可能源于短暂环境问题。因此，每个候选经验条目必须通过两道门控才能注入后续轮次。阈值门控要求至少两个独立运行提供证据；对抗审查门控部署一个专门负责证伪所声明因果机制的智能体。经受住审查的条目标为 confirmed，存在争议的标为 contested（注入时附带警告），被证伪的条目则永久排除。这与 AgenticRecTune (Wu et al., 2026) 中直接从 A/B 结果提炼经验的 Skillhub 不同：结果本身无法解释其为何有效，不能解释自身机制的经验条目会成为负担—它可能泛化不佳，或编码虚假相关并污染后续决策。在知识库

层面要求经验证的因果解释后才能晋级，与单轮实验中要求可证伪归因采用了同一标准。

闭合循环。 这些机制共同构成自进化搜索。每个实验结论都会更新探索记忆：已确认的 **playbook** 条目强化有效方向，反模式触发论文级或模块级剪枝，被证伪的前提则关闭搜索空间的整个分支。因此，下一轮探索从一个不仅更大—每日摄取会持续引入新论文—而且更锐利的搜索空间开始，其中标注了系统历次实验累积的全部判定。

7.3. 讨论

SGPO 将智能体改进视为可离线回放的系统优化问题。它补充了在线实验分析：实验分析解释推荐策略是否有效，而 SGPO 则在轨迹本身暴露反复出现的弱点时，解释智能体 **harness** 应如何改变。当前实现有意采取保守策略：一次只更新一个子智能体规格，冻结运行 **harness** 的其余部分，并仅在配对回放后接纳编辑。这种更新路径比不受约束的自我编辑更慢，但对生产推荐 workflow 更安全，因为一次范围过宽的 **harness** 变更就可能污染下游实验产物。

8. 实验

8.1. 生产测量与闭环性能

为理解智能体如何重塑算法迭代，我们从三个维度研究 AgentX 对工程师迭代的影响：

- **RQ1.** AgentX 是否缩短了从想法到发布的周期？
- **RQ2.** AgentX 是否让每个 **worker** 产出更多发布级结果？
- **RQ3.** AgentX 发布的实验是否带来可测量的线上指标增益？

回答这些问题需要观察完整生命周期。因此，我们构建了一个对每个阶段进行埋点的监控平台，并分析其产生的转化指标。

监控与数据有效性。 为纵向评估 AgentX 自动迭代流水线，我们对所有 **worker** 的实验生命周期进行埋点：实验文档、审查状态、编码与部署事件以及 AB 决策均被规范化为与标准生命周期阶段对齐的仅追加事件日志。相邻转化率在固定快照范围内计算，因此对同一时间窗口的任何重跑都能复现相同的聚合结果和阶段归因。所得节点计数和相邻转化率构成表 5 与式 10 报告的链式漏斗。

我们使用同一组证据回答三个研究问题：在为期三周的部署中，三个 **AgentX worker** 在 **快手 App** 的两个生产场景——主信息流推荐和生活服务商业化——并发运行从想法到发布的闭环。监控平台将闭环中的每个节点—想法通过、编码并上线以及正向评估—记录为显式状态转移，其中正向评估是指符合全量发布条件的实验，即可上线结果（LR）。表 5 报告各节点转化率，表 8 则比较 AgentX 与单名算法工程师的人均生产率。

Table 5 | AgentX 从想法到发布迭代闭环的节点级转化率与计数。

Scenarios	Ideas	Idea Pass	Code-&Launch	Positive Eval.	LR
主信息流	361	27.7%	95.0%	8.4%	8
生活服务	13	46.1%	83.3%	40.0%	2
Overall	374	28.34%	94.3%	9.9%	10

Table 6 | 未通过 VALIDATE 门控的 268 个想法的分类拒绝原因。每个想法均按优先级 $A \rightarrow B \rightarrow C \rightarrow E \rightarrow D \rightarrow F \rightarrow Z$ 唯一归入一个类别。

ID	拒绝类别	比例
A	参数-资源冲突	64.7%
B	前置条件 enable 标志关闭	6.4%
C	违反硬约束、白名单或能力边界	7.5%
E	实现不可行 / 未找到参数 / 缺少前置代码	0.6%
D	与历史实验重叠	5.2%
F	假设薄弱 / 护栏风险 / 空结果风险	1.2%
Z	缺少用户/物品属性	14.5%
总计		100.0%

Table 7 | 编码阶段观察到的分类失败模式。

ID	编码失败类别	比例
CA	调试分支上的 DSL / 强制启用接线错误	35%
CB	违反 C++ / MaTX 编译器约束	20%
CC	并行想法之间的 if/else 分支结构错误	15%
CD	Dryrun 状态机误判	10%
CE	本地验证环境不可用	10%
CF	违反 dryrun-mr 启动契约或工作树不干净	5%
CG	日志解析伪象被误认为代码缺陷	5%
总计		100%

该流水线形成链式转化漏斗，其中每一阶段均以上一阶段成功为条件：

$$\underbrace{374}_{\text{ideas}} \xrightarrow{\text{idea pass } 28.34\%} \underbrace{106}_{\text{passed}} \xrightarrow{\text{code\&launch } 94.3\%} \underbrace{100}_{\text{launched}} \xrightarrow{\text{positive eval. } 9.9\%} \underbrace{10}_{\text{LR}} . \quad (10)$$

在该表述下，两条业务线内部一致： $361 \times 27.7\% \times 95.0\% \times 8.4\% = 8$ （主信息流）和 $13 \times 46.1\% \times 83.3\% \times 40.0\% = 2$ （生活服务）。

哪些因素会在编码和启动之前阻断想法？表 6 将 268 个被拒想法划分为互斥类别，并揭示出显著的不对称性：看似由智能体负责的失败，实际上集中在智能体之外。仅 AB 平台的参数资源冲突就占

全部拒绝 (A) 的 **64.7%**——目标参数已被保留组合实验、进行中实验或另一流量世界占用，因此想法在技术上合理，却无法在生产流量中落地。三类相邻的运维摩擦又贡献了 **12.2%**：先决 enable 标志仍关闭的软开关疏漏 (B, 6.4%)、只会稀释现有流量的历史可启动结果或进行中实验重复项 (D, 5.2%)，以及目标参数不在实时 DSL 世界中的不可行实现 (E, 0.6%)。另有 **14.5%** (Z) 来自数据基础设施：想法依赖尚未暴露在生产特征库中的用户侧或物品侧信号，因此无论智能体推理多么敏锐，都无法接入实验。平台与基础设施约束合计吸收了全部拒绝的 **91.4%**，真正的智能体错误——硬约束或白名单违规 (C, 7.5%) 以及假设薄弱或护栏风险 (F, 1.2%) ——仅占 **8.7%**。由此得到两点启示：闭环瓶颈在运维而非算法——仅弥合 A 类缺口就能挽回约三分之二当前损失的想法；因此，杠杆最高的下一步并非更聪明的智能体，而是一个在头脑风暴之前查询 AB 平台状态的上游冲突检查器，使资源锁定参数从一开始就不会进入候选池。

剩余编码闭环失败集中在哪里？表 7 揭示了与表 6 相同的不对称性：超过 **95%** 的失败来自基础设施侧，而非智能体侧。调试分支上的框架语法违规——DSL 接线 (CA, 35%)、MaTX/C++ 约束 (CB, 20%) 以及 if/else 结构 (CC, 15%) ——占全部编码失败的 **70%**。工具链和环境摩擦 (CD+CE) 又占 **20%**。编码智能体真正的算法错误不足 **5%**。结论与表 6 一致：剩余损失源于可预先固化为模板的框架惯例，而非智能体的推理质量。

AgentX 将串行工作流转化为并行流水线，从而缩短从想法到发布的周期 (RQ1)。人工工作流将生命周期作为单一串行链执行：每个想法都会阻塞下一个，吞吐量受最慢交接环节限制。AgentX 将提案、编码、启动和监控阶段解耦为独立工作单元，把该链条重构为并行流水线，使不同想法可以同时处于不同阶段。因此，每个想法的有效周期缩短为最繁忙阶段的耗时，而非所有阶段耗时之和。实证上，三个工作单元在三周内推动 **374** 个想法走完整个闭环；随着系统通过技能整合、陷阱积累和 dryrun 模板成熟实现自我演化，每个工作单元的并发吞吐量约每周翻倍。在三周窗口内取平均，每个 AgentX 工作单元可维持约 **12** 个并发实验，而传统人工工作流中每名工程师为 **1.5** 个，即单工作单元提升 **8×** (表 8)。由于各阶段不会迫使想法争抢共享资源，增加工作单元可近似线性提升吞吐量，因此迭代速度取决于想法供给和评估带宽，而非某个人推动单个想法走完链条的速度。

AgentX 在人工审查下自动化闭环两端，从而产出更多发布级结果 (RQ2)。在前端，*brainstorm agent* 并行生成候选想法并将其送入审查关口，使工程师从逐条编写想法转为筛选高容量想法流；仅有通过审查的 **28.34%** 得以保留 (表 5)。由此，人工投入从想法生产提升为想法选择。在后端，*developing agent* 是决定性杠杆：把想法转化为通过 dry-run 验证的可部署代码，是人工工作流中劳动最密集、最易失败的环节。*developing agent* 将编码与启动率提升至 **94.3%**，因而承担了几乎全部此类工作，损失的想法很少，无需人工值守，并让工程师专注于审查而非执行。三周内，三个工作单元将 **374** 个想法转化为 **10** 个可启动结果（端到端转化率 $10/374 \approx 2.67\%$ ），即每个工作单元约 **3.3** 个 LR。我们并不声称单想法质量相当：如表 8 所示，资深工程师的单想法命中率仍高约 **1.9×**，因为人工想法经过精心选择和预筛选。AgentX 有意以自动化下的规模换取稀缺条件下的精度：按工作单元周统计，其累计 app 使用时长增益为 **0.0623%**，工程师则为 **0.0167%**，即单位人力容量实现的业务价值提升 **3.7×**。因此，自动化闭环两端是迈向算法迭代工作流全面自动化的具体一步。

Table 8 | 传统人工迭代工作流下，AgentX 工作单元与算法工程师的单工作单元生产率和并发度对比。

指标（每位工作者·周）	AgentX	工程师	比率
并发实验数	12	1.5	8×
LR 数量	1.1	0.08	13.8×
产生的累计应用时长增益	0.0623%	0.0167%	3.7×
单想法上线转化率	2.7%	5.1%	0.53×

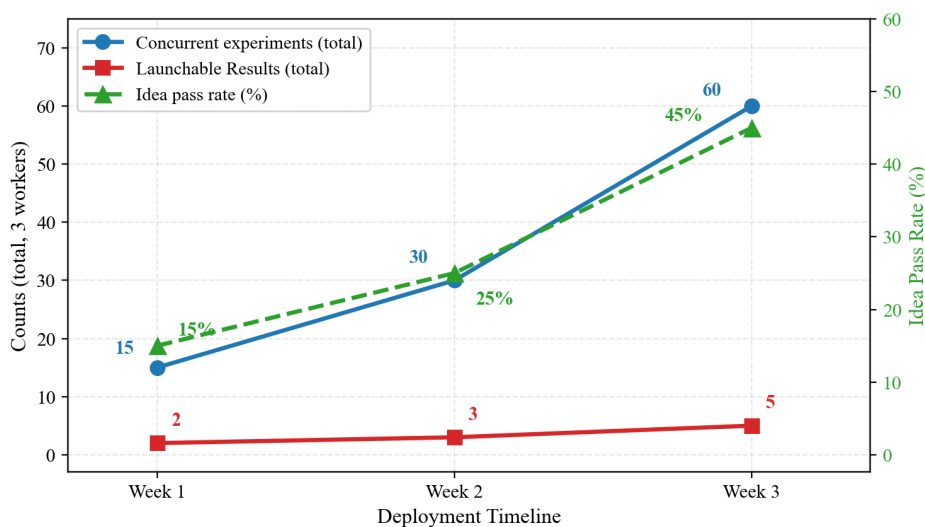


Figure 10 | 三周自我演化重塑了 AgentX 闭环：每周并发实验数增至四倍（15→60），想法通过率增至三倍（15%→45%），每周可启动结果数增加超过一倍（2→5）。技能整合、陷阱积累和 dryrun 模板成熟使吞吐量与选择性形成复利——系统不再只是产出更多，而是产出更多正确的想法。

AgentX 发布的实验带来了可衡量的线上增益，其规模随算力扩展（RQ3）。系统端到端产出了 10 个可启动结果——361 个主信息流想法贡献 8 个，13 个生活服务想法贡献 2 个——这些发布转化为可观的业务收益：

- **主信息流。**快手 App 用户应用消费时长累计提升 **0.561%**。
- **生活服务。**为快手平台带来超过人民币 **1 亿元**的年化收入。

如图 10 所示，在三周窗口内，自我演化同时扩大了吞吐量并提高了选择性。由于人力仅用于审查和决策，而所有执行密集型阶段均已自动化，发布级结果的数量不再受人数限制，而主要取决于分配给闭环的算力。吞吐量随工作单元数量近似线性扩展——按此处观测，每增加一个工作单元，每周约贡献 1.1 个 LR——在爬坡阶段，系统的自我演化还使其每周额外**翻倍**。因此，本研究展示的业务价值可沿两个互补维度扩展：更多工作单元（随算力线性增长）和更成熟的系统（早期超线性增长）。

8.2. AgentX 的模型迭代漏斗

我们对 AgentX 驱动的推荐模型研发开展了一项独立研究。该研究聚焦使用 AgentX 实现端到端模型迭代 workflow，包括自主论文复现、模块消融、跨论文架构组合和新架构发现。需要强调的是，该研究不属于表 5 总结的三周生产部署；其节点计数和转化率基于独立实验池计算，因此与表 5 不可直接比较。

研究设置。 我们整理了一批近期推荐论文作为候选池，要求 AgentX 按照附录 B 的同一协议，在统一代码库上复现每篇论文。随后，根据复现模型在 KuaiRand (Gao et al., 2022)、Taobao (Zhu et al., 2018)、Amazon (Ni et al., 2019) 和 ML-1M (Harper and Konstan, 2015) 四个公开数据集上的聚合性能进行排序。表现最佳的复现模型被选为跨论文架构组合的种子集。对于每篇种子论文，AgentX 会自动从其他种子论文中识别当前种子模型尚未实现的互补模块，再通过 LLM 驱动的代码组合将这些模块合成到种子架构中。每个组合模型都注册为新实验并重新进入同一排行榜，成功的组合因而可以成为后续迭代轮次的候选种子。

在整个研究中，比较基线是当前部署于快手 App 的生产排序算法，仅为适配公开数据集特征 schema 而进行了简化。该简化移除了业务专属特征和服务约束，同时保留核心架构和训练配方。因此，此设置中“超过基线”的实验应被理解为潜在业务价值的必要但非充分指标。

漏斗。 模型智能体闭环遵循两阶段漏斗。第一阶段为筛选阶段，AgentX 将大量复现论文提炼为更小的高质量种子集。第二阶段为扩展与验证阶段，AgentX 自动派发跨论文架构组合，并通过离线评估、线上测试和上线审查门控进行筛选。详细结果见表 9。

Table 9 | *model-agent* 迭代漏斗的节点级计数与相邻转化率。这是一项专门的独立研究，并不属于表 5 所报告的三周生产部署；两个漏斗基于互不相交的实验池计算，不可直接比较。基线是快手 App 的生产排序算法，仅为适配公开数据集的特征模式而做了简化。

阶段	数量	相邻 转化率
候选论文（已复现）	113	—
自动派发实验	214	—
完成运行	180	84.1%
超过基线（公开数据集）	104	57.8%
进入线上 A/B	22	21.2%
线上 A/B 正向	5	22.7%
可上线结果（LR）	2	40.0%

发现。 模型漏斗与表 5 中的策略漏斗有三点不同。第一，**先扩展后剪枝**的形态反映了 AgentX 如何利用离线实验降低线上探索成本。AgentX 从种子模型出发，先将搜索空间扩展为 214 个跨论文架构组合，再通过运行完成检查、离线基准性能和业务侧派发标准过滤潜力不足的候选项。最终仅 22

个实验进入线上 A/B 测试，在消耗生产流量前将线上测试负载降低了近一个数量级。

第二，57.8% 的**超过基线率**应结合设置中所述的基线简化来解释。由于公开数据集基线移除了生产算法所依赖的业务专属特征和服务约束，该比率并不意味着候选模型超过了完整部署的生产系统，而是说明候选项是值得考虑进行生产集成的有效模块或架构。

第三，漏斗右侧 **22 → 5 → 2** 的收缩最为显著。即使观察到离线增益，线上正向变化与上线审查批准的联合要求仍使候选集缩小约一个数量级。这与生产漏斗 9.9% 的正向评估率一致，表明模型侧 LR 产出的主要瓶颈是线上审查门控，而非离线准确率或编码吞吐量。

结论。 这项独立的模型智能体研究表明，AgentX 的模型迭代组件能够完成从论文复现、架构组合到离线评估、线上测试和可上线结果的端到端闭环。在互不重叠的实验面上，系统从原始论文池和自动派发实验池中均产出 LR 级制品：113 篇候选论文中有 2 篇、214 个组合实验中有 2 个达到 LR。两个 LR 模型使快手 App 直播时长提升了 **0.865%**。

结合表 5 的生产漏斗，该结果说明 AgentX 并不局限于单一研究面，而是实例化了一种可移植的闭环模式，可同时用于策略侧和模型侧推荐系统迭代。本闭环使用的逐方法详细结果、模块组合研究和工业场景迁移见附录 B。

8.3. 案例展示 I：端到端 AgentX 实验

当前 AgentX 循环最有力的展示并非某个孤立指标，而是多个上线评审文档均可追溯到一条自主研究路径：提案生成、可行性检查、实现、在线实验和实验后分析。这表明 AgentX 已能在真实推荐场景中完成端到端循环。

PCV 增强的约束精排分数。 该上线评审案例展示 AgentX 如何通过闭环多智能体循环改进机制级排序想法。首要目标是在保持用户真人内容指标稳定的同时，提高快手主信息流中的用户观看时长。为此，AgentX 引入消费后价值（PCV）作为额外排序信号。PCV 捕获视频消费后的分享、收藏和重播等行为，能够反映即时观看时长或点击信号之外的持久内容价值。然而 PCV 也有风险：高 PCV 并不总意味着高内容质量，因为低质病毒式内容或诱饵内容也可能触发消费后行为。下面介绍该案例完整的 AgentX 实验流程。

Loop 1：直接提升 PCV。 如表 10 所示，第一轮测试 PCV 能否直接用作额外排序信号。Brainstorm Agent 生成多个候选方向并选择提升 PCV；Developing Agent 将其转化为简单的乘法评分公式：

$$S_1 = B_r \cdot (1 + \beta P), \quad (11)$$

其中， B_r 表示面向相关性的基础分， P 表示混合 PCV 分数， β 为固定 PCV 权重。随后 Evaluation Agent 分析线上 A/B 结果。第一版在主要时长指标上取得正向但较弱的增益：人均观看时长提升 +0.034%，用户观看时长提升 +0.021%。然而，多项诊断指标显示潜在不稳定性：活跃设备略微负向（-0.023%），18–30 岁人群的设备平均使用时长出现负向变化（-0.032%）。有效兴趣数、新颖性簇和惊喜度簇等多样性指标也有小幅下降。

Table 10 | 循环 1：直接提升 PCV——智能体输入与输出。

智能体	输入	输出
Brainstorm Agent	排序目标：提升用户观看时长，同时保持真人内容和用户体验护栏稳定。	生成五个候选想法：(1) 会话预算感知评分，(2) 时长偏好匹配，(3) 探索—利用平衡，(4) 负反馈敏感的用户保护，以及 (5) 消费后价值提升。智能体选择消费后价值提升作为上线候选，依据是现有精排分数未充分利用反映长尾内容价值的消费后行为。
Developing Agent	选定的 PCV 提升想法。	实现生产可行的乘法公式： $S_1 = B_r \cdot (1 + \beta P)$ ，由实验开关保护，并可用于在线 A/B 评估。
Evaluation Agent	S_1 的在线 A/B 结果。	给出首轮诊断：时长增益微弱为正但统计上不可靠（人均观看时长 +0.034%；用户观看时长 +0.021%），同时存在活跃设备 -0.023%、18–30 岁人群设备平均使用时长 -0.032% 等诊断风险。结论：直接提升 PCV 噪声过大，需要质量和时长约束。

Evaluation Agent 因此诊断认为，直接提升 PCV 的方向虽有前景，但噪声过大。可能原因是第一版提升了所有高 PCV 内容，未区分高质量消费后价值与噪声或诱导式消费后行为；同时它采用面向相关性的基础分和固定 PCV 权重，既未显式保护观看时长目标，也无法适应用户活动度。这一诊断成为第二轮输入：只有在显式质量门控、活动度感知加权和面向时长约束下才应引入 PCV。

Loop 2：受约束的 PCV 排序。如表 11 所示，第二轮将首轮评估作为输入。Brainstorm Agent 将想法从直接提升 PCV 完善为受约束的 PCV 排序。Developing Agent 实现受约束公式：

$$S_2 = B_d \cdot (1 + \beta(u)G(P)), \quad (12)$$

其中， B_d 表示面向时长的基础分， $G(P) = \max(P - \tau, 0)$ 是阈值为 τ 的质量门控 PCV 信号， $\beta(u)$ 是由用户活动度决定的活动度感知动态权重。Evaluation Agent 随后通过线上 A/B 测试验证第二轮设计。受约束 PCV 机制取得了更明确的正向提升：用户观看时长增加 +0.071%，real-show 增加 +0.118%，同时用户体验护栏保持稳定。Evaluation Agent 还将两轮学习整合为可复用知识制品：直接提升 PCV 可能噪声较大；当 PCV 经质量过滤、按用户活动度缩放并锚定到面向时长的基础分时，受约束 PCV 排序更为可靠。

该案例表明，AgentX 并非一次性想法生成器。它完成了从想法生成、实现、线上评估、反馈驱动的重新设计、知识整合到可测量线上影响的闭环。

8.4. 案例展示 II：与专家智能体协同演进

除独立的端到端模式外，我们还在生活服务场景部署了一种多智能体协作模式，由一个专家智能体与 AgentX 协同工作。具体而言，我们为该场景训练了一个专用推荐决策智能体作为专家智能体。它通过观察用户行为和画像进行用户级诊断，评估当前曝光是否符合用户更深层的服务兴趣，并以自

Table 11 | 循环 2：约束 PCV 排序——智能体输入与输出。

智能体	输入	输出
Brainstorm Agent	Loop-1 评估反馈：直接提升 PCV 的方向为正，但稳健性不足。	将想法完善为受约束的 PCV 排序，并提出三项设计原则：质量门控、活动度感知动态加权 and 面向时长的基础分。
Developing Agent	受约束的 PCV 排序想法。	实现受约束的评分机制： $S_2 = B_d \cdot (1 + \beta(u)G(P))$ ，并准备第二轮线上 A/B 测试。
Evaluation Agent	S_2 的线上 A/B 结果。	产出可供上线审查的结论，并将两轮学习整合为可复用知识制品：直接提升 PCV 噪声较大，而带有质量门控、活动度感知加权和面向时长基础分的受约束 PCV 能带来更清晰的增益。最终线上结果显示用户观看时长 +0.071%、real-show +0.118%，同时用户体验护栏保持稳定。

然语言输出控制决策。随后，AgentX 推理如何处理每项诊断及其在生产中的可行性，将其完善为更合理的方案，编写完整且可部署的执行计划并上线。

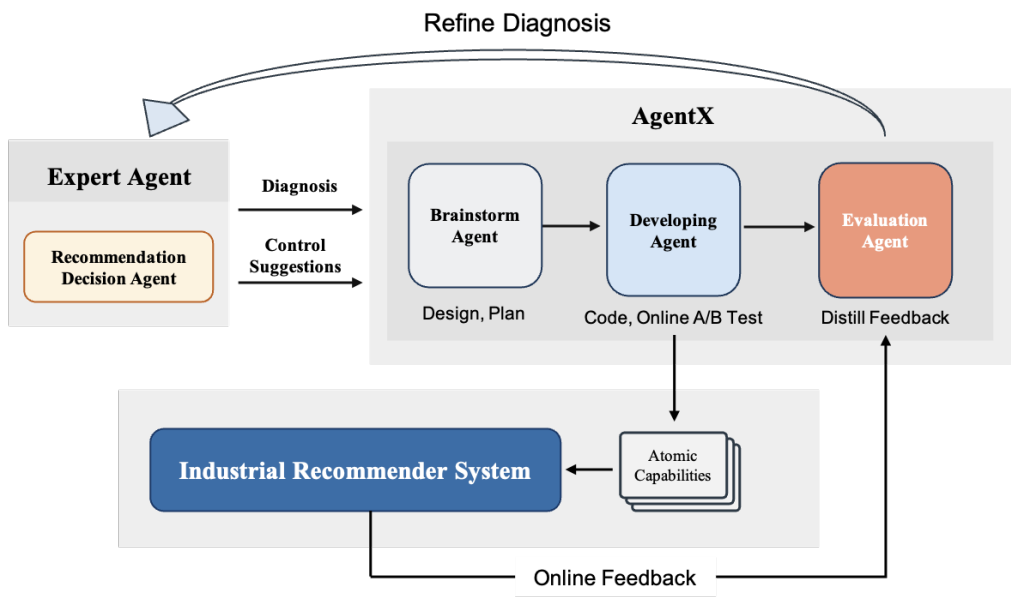


Figure 11 | 生活服务推荐决策智能体的生产流程。AgentX 将诊断信号转化为生产算子，而所得算子扩展了决策智能体可用的原子动作空间。

AgentX 内部的多个子智能体共同支撑这条部署流水线。Brainstorm Agent 评估推荐决策智能体所生成控制建议的可行性，设计具体的原子控制，并确定最终实现方案。Developing Agent 随后将方案实现为可控的线上动作，推动其从代码开发走向部署。Evaluation Agent 读取线上反馈，并将结果提炼为下一轮决策策略。图 11 概览了从专家智能体诊断到 AgentX 部署与线上反馈闭环的多智能体协作流水线。表 12 总结了其中一次闭环中各智能体的输入和输出。

Table 12 | 专家智能体与 AgentX 的协同演化闭环——智能体输入与输出。**收入**指快手平台在线广告系统产生的商业收入。

智能体	输入	输出
推荐决策智能体(专家)	历史序列、用户画像和聚合统计。	通过思维链推理诊断用户的广告兴趣——例如，将司机职业与车辆价格搜索相结合，推断用户对汽车广告具有明确但未被充分满足的兴趣——并以结构化 JSON 输出跨多个因子（行业过滤器、CPM 阈值、CPM 提升比例等）的控制建议。
Brainstorm Agent	推荐决策智能体给出的控制建议。	评估控制建议的可行性，选择 CPM 提升作为控制动作，并为其生成完整、可执行的方案。
Developing Agent	CPM 提升方案。	实现数据获取逻辑和 UV 级控制，推动方案从代码开发走向线上 A/B 测试。
Evaluation Agent	CPM 提升方案的线上 A/B 结果。	产出可供发布审查的结论：UV 级 CPM 提升取得正向效果，使收入增加 +4.7%。

这种协同演化已在生产中展现价值。在一条生活服务记录中，推荐决策智能体在大规模用户群体上取得了 +4.7% 的相对收入提升。这些结果表明，AgentX 的价值不仅在于自主完成端到端研究，还在于充当整个推荐流水线的放大器。对于任何形式的请求，它都能快速推理出可执行方案，推动方案全面部署，并持续监控。在此过程中，推荐决策智能体与 AgentX 沿两个互补维度协同演化：基于生活服务领域知识微调的推荐决策智能体充当领域专家，提供高质量诊断并加深研究闭环；AgentX 则持续提出、实现和验证新的策略方向，将这些诊断落实为细粒度原子动作，从而拓宽闭环。高质量执行轨迹还通过 SGPO 进一步优化 AgentX，使能力在多轮迭代中持续累积。

9. 结论

在本技术报告中，我们介绍了 AgentX：一个通过智能体闭环驱动工业推荐系统端到端迭代的自动化开发框架。AgentX 将生产生命周期分解为角色专门化模块，包括生成基于证据的实验提案的 Brainstorm Agent、将提案转化为生产就绪代码的 Developing Agent，以及负责安全发布和线上 A/B 判断的 Evaluation Agent。在这些模块之外，轨迹驱动的框架演化层会将执行轨迹提炼为对智能体提示词、工作流和底层基础模型的更新，使系统在每一轮迭代中持续增强，而不是重置归零。AgentX 以真实线上 A/B 反馈而非离线指标为依据，在为期三周的生产窗口内，使实验吞吐量和累计线上收益均实现了数量级提升。

除这些具体成果外，AgentX 更深层的意义在于重塑推荐系统迭代的生产函数。长期以来，推荐自动化的吞吐量一直受制于一个结构性事实：高质量想法依赖算法工程师的人力和灵感，而这类资源呈线性增长、稀缺且难以复制。当智能体大规模接管想法生成、代码落地和初步验证后，工程师的角色便从亲自执行每项实验，转向界定哪些问题值得探索以及设计更高效的智能体系统。轨迹数据将这一转变贯通起来，因为底层系统的每次改进都会自动由后续所有实验继承。由此，工业推荐

系统的迭代从人力的线性叠加，被改写为一种可复利、可工程化的杠杆。我们发布本报告，旨在邀请更广泛的社区在各自系统中探索智能体驱动的迭代。

References

- Thornton, C., F. Hutter, H. H. Hoos, and K. Leyton-Brown (2013). “Auto-WEKA: Combined selection and hyperparameter optimization of classification algorithms”. In: *Proceedings of the 19th ACM SIGKDD international conference on Knowledge discovery and data mining*, pp. 847–855.
- Harper, F. M. and J. A. Konstan (2015). “The movielens datasets: History and context”. In: *Acm transactions on interactive intelligent systems (tiis)* 5.4, pp. 1–19.
- Xu, Y., N. Chen, A. Fernandez, O. Sinno, and A. Bhasin (2015). “From infrastructure to culture: A/B testing challenges in large scale social networks”. In: *Proceedings of the 21th ACM SIGKDD international conference on knowledge discovery and data mining*, pp. 2227–2236.
- Guo, H., R. Tang, Y. Ye, Z. Li, and X. He (2017). “DeepFM: a factorization-machine based neural network for CTR prediction”. In: *arXiv preprint arXiv:1703.04247*.
- Fabijan, A., P. Dmitriev, H. H. Olsson, and J. Bosch (2018). “Online controlled experimentation at scale: an empirical survey on the current state of A/B testing”. In: *2018 44th Euromicro Conference on Software Engineering and Advanced Applications (SEAA)*. IEEE, pp. 68–72.
- Zhou, G., X. Zhu, C. Song, Y. Fan, H. Zhu, X. Ma, Y. Yan, J. Jin, H. Li, and K. Gai (2018). “Deep interest network for click-through rate prediction”. In: *Proceedings of the 24th ACM SIGKDD international conference on knowledge discovery & data mining*, pp. 1059–1068.
- Zhu, H., X. Li, P. Zhang, G. Li, J. He, H. Li, and K. Gai (2018). “Learning tree-based deep model for recommender systems”. In: *Proceedings of the 24th ACM SIGKDD international conference on knowledge discovery & data mining*, pp. 1079–1088.
- Ni, J., J. Li, and J. McAuley (2019). “Justifying recommendations using distantly-labeled reviews and fine-grained aspects”. In: *Proceedings of the 2019 conference on empirical methods in natural language processing and the 9th international joint conference on natural language processing (EMNLP-IJCNLP)*, pp. 188–197.
- Liu, B., C. Zhu, G. Li, W. Zhang, J. Lai, R. Tang, X. He, Z. Li, and Y. Yu (2020). “Autofis: Automatic feature interaction selection in factorization models for click-through rate prediction”. In: *proceedings of the 26th ACM SIGKDD international conference on knowledge discovery & data mining*, pp. 2636–2645.
- Song, Q., D. Cheng, H. Zhou, J. Yang, Y. Tian, and X. Hu (2020). “Towards automated neural interaction discovery for click-through rate prediction”. In: *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, pp. 945–955.
- Gao, C., S. Li, Y. Zhang, J. Chen, B. Li, W. Lei, P. Jiang, and X. He (2022). “Kuairand: An unbiased sequential recommendation dataset with randomly exposed videos”. In: *Proceedings of the 31st ACM international conference on information & knowledge management*, pp. 3953–3957.

- Achiam, J., S. Adler, S. Agarwal, L. Ahmad, I. Akkaya, F. L. Aleman, D. Almeida, J. Altenschmidt, S. Altman, S. Anadkat, et al. (2023). “Gpt-4 technical report”. In: *arXiv preprint arXiv:2303.08774*.
- Hollmann, N., S. Müller, and F. Hutter (2023). “Large language models for automated data science: Introducing caafe for context-aware automated feature engineering”. In: *Advances in Neural Information Processing Systems* 36, pp. 44753–44775.
- Zhang, T., D. Cheng, Y. He, Z. Chen, X. Dai, L. Xiong, F. Yan, H. Li, Y. Chen, and W. Wen (2023). “NASRec: weight sharing neural architecture search for recommender systems”. In: *Proceedings of the ACM Web Conference 2023*, pp. 1199–1207.
- Cao, J., S. Wang, Y. Li, S. Wang, J. Tang, S. Wang, S. Yang, Z. Liu, and G. Zhou (2024). “Moment&Cross: Next-Generation Real-Time Cross-Domain CTR Prediction for Live-Streaming Recommendation at Kuaishou”. In: *arXiv preprint arXiv:2408.05709*.
- Guo, S., C. Deng, Y. Wen, H. Chen, Y. Chang, and J. Wang (2024). “Ds-agent: Automated data science by empowering large language models with case-based reasoning”. In: *arXiv preprint arXiv:2402.17453*.
- Liu, A., B. Feng, B. Xue, B. Wang, B. Wu, C. Lu, C. Zhao, C. Deng, C. Zhang, C. Ruan, et al. (2024). “Deepseek-v3 technical report”. In: *arXiv preprint arXiv:2412.19437*.
- Lu, C., C. Lu, R. T. Lange, J. Foerster, J. Clune, and D. Ha (2024). “The ai scientist: Towards fully automated open-ended scientific discovery”. In: *arXiv preprint arXiv:2408.06292*.
- Team, G., M. Riviere, S. Pathak, P. G. Sessa, C. Hardin, S. Bhupatiraju, L. Hussenot, T. Mesnard, B. Shahriari, A. Ramé, et al. (2024). “Gemma 2: Improving open language models at a practical size”. In: *arXiv preprint arXiv:2408.00118*.
- Trirat, P., W. Jeong, and S. J. Hwang (2024). “Automl-agent: A multi-agent llm framework for full-pipeline automl”. In: *arXiv preprint arXiv:2410.02958*.
- Zhai, J., L. Liao, X. Liu, Y. Wang, R. Li, X. Cao, L. Gao, Z. Gong, F. Gu, M. He, et al. (2024). “Actions speak louder than words: Trillion-parameter sequential transducers for generative recommendations”. In: *arXiv preprint arXiv:2402.17152*.
- Zhang, B., L. Luo, Y. Chen, J. Nie, X. Liu, D. Guo, Y. Zhao, S. Li, Y. Hao, Y. Yao, et al. (2024). “Wukong: Towards a scaling law for large-scale recommendation”. In: *arXiv preprint arXiv:2403.02545*.
- Deng, J., S. Wang, K. Cai, L. Ren, Q. Hu, W. Ding, Q. Luo, and G. Zhou (2025). “Onerec: Unifying retrieve and rank with generative recommender and iterative preference alignment”. In: *arXiv preprint arXiv:2502.18965*.
- Hong, S., Y. Lin, B. Liu, B. Liu, B. Wu, C. Zhang, D. Li, J. Chen, J. Zhang, J. Wang, et al. (2025). “Data interpreter: An llm agent for data science”. In: *Findings of the Association for Computational Linguistics: ACL 2025*, pp. 19796–19821.
- Luo, X., J. Cao, T. Sun, J. Yu, R. Huang, W. Yuan, H. Lin, Y. Zheng, S. Wang, Q. Hu, et al. (2025). “Qarm: Quantitative alignment multi-modal recommendation at kuaishou”. In: *Proceedings of the 34th ACM International Conference on Information and Knowledge Management*, pp. 5915–5922.

- Ou, Y., Y. Luo, J. Zheng, L. Wei, Z. Yu, S. Qiao, J. Zhang, D. Zheng, Y. Mao, Y. Gao, et al. (2025). “Automind: Adaptive knowledgeable agent for automated data science”. In: *arXiv preprint arXiv:2506.10974*.
- Schmidgall, S., Y. Su, Z. Wang, X. Sun, J. Wu, X. Yu, J. Liu, M. Moor, Z. Liu, and E. Barsoum (2025). “Agent laboratory: Using llm agents as research assistants”. In: *Findings of the Association for Computational Linguistics: EMNLP 2025*, pp. 5977–6043.
- Team, K., Y. Bai, Y. Bao, Y. Charles, C. Chen, G. Chen, H. Chen, H. Chen, J. Chen, N. Chen, et al. (2025). “Kimi k2: Open agentic intelligence”. In: *arXiv preprint arXiv:2507.20534*.
- Xu, X., H. Wang, W. Guo, L. Zhang, W. Yang, R. Yu, Y. Liu, D. Lian, and E. Chen (2025). “Multi-granularity interest retrieval and refinement network for long-term user behavior modeling in ctr prediction”. In: *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V. 1*, pp. 2745–2755.
- Xue, E., K. Chen, Z. Huang, Y. Ji, and H. Wang (2025). “Improve: Iterative model pipeline refinement and optimization leveraging LLM experts”. In: *arXiv preprint arXiv:2502.18530*.
- Yamada, Y., R. T. Lange, C. Lu, S. Hu, C. Lu, J. Foerster, J. Clune, and D. Ha (2025). “The ai scientist-v2: Workshop-level automated scientific discovery via agentic tree search”. In: *arXiv preprint arXiv:2504.08066*.
- Yang, A., A. Li, B. Yang, B. Zhang, B. Hui, B. Zheng, B. Yu, C. Gao, C. Huang, C. Lv, et al. (2025a). “Qwen3 technical report”. In: *arXiv preprint arXiv:2505.09388*.
- Yang, X., X. Yang, S. Fang, Y. Zhang, J. Wang, B. Xian, Q. Li, J. Li, M. Xu, Y. Li, et al. (2025b). “R&D-Agent: An LLM-Agent Framework Towards Autonomous Data Science”. In: *arXiv preprint arXiv:2505.14738*.
- Zhu, J., Z. Fan, X. Zhu, Y. Jiang, H. Wang, X. Han, H. Ding, X. Wang, W. Zhao, Z. Gong, et al. (2025). “Rankmixer: Scaling up ranking models in industrial recommenders”. In: *Proceedings of the 34th ACM International Conference on Information and Knowledge Management*, pp. 6309–6316.
- Cao, Y., A. Ranjan, S. Subramaniam, E. Korpeoglu, K. Nag, and K. Achan (2026). “CASE: Cadence-Aware Set Encoding for Large-Scale Next Basket Repurchase Recommendation”. In: *arXiv preprint arXiv:2604.06718*.
- Chen, J., S. Zhang, B. Hu, C. Zhou, J. Pan, G. Xue, W. Ning, G. Weng, W. Zheng, S. Liu, et al. (2026). “RankUp: Towards High-rank Representations for Large Scale Advertising Recommender Systems”. In: *arXiv preprint arXiv:2604.17878*.
- Du, S., X. Yan, J. Shi, Z. Cao, S. Feng, Z. Liang, B. Sun, T. Peng, Y. Zhou, X. Li, et al. (2026). “MLEvolve: A Self-Evolving Framework for Automated Machine Learning Algorithm Discovery”. In: *arXiv preprint arXiv:2606.06473*.
- Huang, X., H. Zhang, Z. Fan, Y. Huang, Z. Wei, Z. Chai, J. Ni, Y. Zheng, and Q. Chen (2026). “MixFormer: Co-Scaling Up Dense and Sequence in Industrial Recommenders”. In: *arXiv preprint arXiv:2602.14110*.

- Kim, S., S. Park, H. Kang, W. Kim, J. Seo, Y. In, K. Yoon, and C. Park (2026). “Self-EvolveRec: Self-Evolving Recommender Systems with LLM-based Directional Feedback”. In: *arXiv preprint arXiv:2602.12612*.
- Lin, T., Z. Xiong, B. Ou, Y. Qin, L. Xu, X. Zhong, Y. Hu, Z. Wang, T. Zhou, Y. Xu, et al. (2026). “LASER: An Efficient Target-Aware Segmented Attention Framework for End-to-End Long Sequence Modeling”. In: *arXiv preprint arXiv:2602.11562*.
- Nam, J., J. Yoon, J. Chen, J. Shin, S. Arik, and T. Pfister (2026). “Mle-star: Machine learning engineering agent via search and targeted refinement”. In: *Advances in Neural Information Processing Systems* 38, pp. 116692–116712.
- Tang, J., L. Xia, Z. Li, and C. Huang (2026). “Ai-researcher: Autonomous scientific innovation”. In: *Advances in Neural Information Processing Systems* 38, pp. 9481–9520.
- Wang, C., B. Wu, T. Pang, J. Wang, J. Yang, J. Liu, H. Zhang, H. Zhu, L. Shen, S. Wang, et al. (2026a). “SORT: A Systematically Optimized Ranking Transformer for Industrial-scale Recommenders”. In: *arXiv preprint arXiv:2603.03988*.
- Wang, H., Y. Wu, D. Chang, L. Wei, and L. Heldt (2026b). “Self-evolving recommendation system: End-to-end autonomous model optimization with LLM agents”. In: *arXiv preprint arXiv:2602.10226*.
- Wang, S., Y. Huang, R. Yang, S. Wen, P. Xu, J. Cao, Y. Liu, K. Cai, C. Guo, S. Wang, et al. (2026c). “OneLive: Dynamically Unified Generative Framework for Live-Streaming Recommendation”. In: *arXiv preprint arXiv:2602.08612*.
- Wu, X., Y. Zhuan, R. Wei, H. Chen, D. Bai, J. Liu, X. Wang, X. Wang, L. Wang, and X. Cheng (2026). “AgenticRecTune: Multi-Agent with Self-Evolving Skillhub for Recommendation System Optimization”. In: *arXiv preprint arXiv:2604.26969*.
- Yu, Y., S. Qiao, L. Shen, B. Wang, and X. Zeng (2026). “Beyond Dense Connectivity: Explicit Sparsity for Scalable Recommendation”. In: *arXiv preprint arXiv:2604.08011*.
- Yuan, K., J. Bi, D. Cheng, C. Wu, S. Xiao, B. Cao, J. Wu, and Y. Jiang (2026). “HiSAC: Hierarchical Sparse Activation Compression for Ultra-long Sequence Modeling in Recommenders”. In: *arXiv preprint arXiv:2602.21009*.
- Zeng, A., X. Lv, Z. Hou, Z. Du, Q. Zheng, B. Chen, D. Yin, C. Ge, C. Huang, C. Xie, et al. (2026). “Glm-5: from vibe coding to agentic engineering”. In: *arXiv preprint arXiv:2602.15763*.
- Zhang, R., Z. Huang, Z. Wang, K. Sun, B. Zheng, Y. Jiang, Z. Chen, Z. Ouyang, H. Xie, P. Shen, et al. (2026a). “Zenith: Scaling up Ranking Models for Billion-scale Livestreaming Recommendation”. In: *arXiv preprint arXiv:2601.21285*.
- Zhang, W., X. Li, Q. Ge, K. Dong, P. Jia, X. Li, Z. Zhang, M. Wang, Y. Wang, H. Guo, et al. (2026b). “Exploring Recommender System Evaluation: A Multi-Modal LLM Agent Framework for A/B Testing”. In: *Proceedings of the 32nd ACM SIGKDD Conference on Knowledge Discovery and Data Mining V. 1*, pp. 2878–2889.

Zhang, Y., S. Dong, X. Wang, B. Chen, Q. Jia, S. Wang, J. Jiao, R. Li, J. Liu, C. Ma, et al. (2026c). “SMES: Towards Scalable Multi-Task Recommendation via Expert Sparsity”. In: *arXiv preprint arXiv:2602.09386*.

A. 作者名单

核心贡献者 Changxin Lao, Fei Pan, Guozhuang Ma, Han Li, Huihuang Lin, Jijun Shi, Kangzhi Zhao, Kun Gai, Mo Zhou, Qinqin Zhou, Quan Chen, Ruochen Yang, Shifu Bie, Shijie Yi, Shuang Yang, Shuo Yang, Wenhao Li, Wentao Xie, Xiao Lv, Xuming Wang, Yijun Wang, Yiming Chen, Yusheng Huang, Zhongyuan Wang, Zibo Zhao, Zijie Zhuang

贡献者 Baoning Xia, Chao Liu, Chaoyi Ma, Chubo He, Dawei Cong, Feng Jiang, Gang Wang, Guilin Xia, Hanwen Xu, Jiahong Xie, Jiahui Qiao, Jian Liang, Jiangfan Yue, Jing Wang, Jinghan Yang, Jinghui Jia, Kan Qin, Lei Wang, Ming Li, Peilin Song, Pengbo Xu, Qiang Luo, Ruiming Tang, Shiyang Liu, Shuxian Jin, Tao Wang*, Tao Zhang, Xiang Gao, Xianghan Li, Yingsong Luo, Yiwen Ning, Yongcheng Liu, Yueyang Liu, Yuan Guo, Zhaojie Liu, Zhenkai Cui

所有作者均按名字的字母顺序排列。 * 表示已离开团队的成员。

B. 模型研究实验

B.1. 复现实验

Table 13 | 多种方法下复现模型的性能比较。基线结果以**粗体**表示，最优结果以下划线表示。

模型	KuaiRand		Taobao		Amazon		ML-1M	
	AUC	提升 ↑	AUC	提升 ↑	AUC	提升 ↑	AUC	提升 ↑
RankMixer	0.6860	-	0.5647	-	0.6671	-	0.7935	-
CASE	0.6643	-0.0217	0.6090	+0.0443	<u>0.6913</u>	<u>+0.0242</u>	0.7947	+0.0012
HiSAC	0.6860	+0.0000	0.6099	+0.0452	0.6671	+0.0000	0.7935	+0.0000
SORT	0.6523	-0.0337	<u>0.6189</u>	<u>+0.0542</u>	0.6770	+0.0099	<u>0.8020</u>	<u>+0.0085</u>
LASER	0.6860	+0.0000	0.5881	+0.0234	0.6671	+0.0000	0.7935	+0.0000
Zenith	0.6895	+0.0035	0.5821	+0.0174	0.6732	+0.0061	0.7864	-0.0071
RankUp	0.6742	-0.0118	0.5780	+0.0133	0.6808	+0.0137	0.7948	+0.0013
Wukong	<u>0.6897</u>	<u>+0.0037</u>	0.5685	+0.0038	0.6588	-0.0083	0.7881	-0.0054
QARM	0.6879	+0.0019	0.5712	+0.0065	0.6690	+0.0019	0.7936	+0.0001

自主模型探索的一个前提，是能够忠实复现近期发表的推荐方法。因此，我们从这一基础角度评估 AgentX，考察其将论文转化为可执行、可复现工程产物的能力。具体而言，AgentX 需要阅读每篇目标论文，将其中提出的机制分解为可复用组件，在统一的骨干架构之上实现这些组件，并按照统一协议开展训练和评估。我们使用 RankMixer (Zhu et al., 2025) 作为基础模型，并在四个公开推荐数据集 KuaiRand (Gao et al., 2022)、Taobao (Zhu et al., 2018)、Amazon (Ni et al., 2019) 和 ML-

1M (Harper and Konstan, 2015) 上进行评估。复现的方法包括 CASE (Cao et al., 2026)、HiSAC (Yuan et al., 2026)、SORT (Wang et al., 2026a)、LASER (Lin et al., 2026)、Zenith (Zhang et al., 2026a)、RankUp (Chen et al., 2026)、Wukong (Zhang et al., 2024) 和 QARM (Luo et al., 2025) 等。我们以 AUC 为主要评估指标，并在表 13 中展示结果。

结果表明，AgentX 针对每个（方法，数据集）组合都完成了完整复现闭环，在同一 RankMixer 骨干上针对异构机制产出了 32 个可训练变体。这说明 AgentX 能够在统一协议下，将多样化的论文级设计忠实转化为可运行产物。

此外，没有任何单一方法在全部四个数据集上占据主导地位。AgentX 能够揭示这些跨数据集差异，而非报告一致为正的数值，这表明其复现结果反映了方法的真实行为，并为后续模块探索闭环提供了可信的构建模块。

B.2. 模块探索实验

为评估 AgentX 能否基于既有研究和近期研究知识，自主发现并实现对推荐模型有效的改进，我们开展了离线自探索实验。具体而言，我们仍以 RankMixer 为基础模型，并在 KuaiRand 和 Taobao 两个公开推荐数据集上进行评估。AgentX 被要求自动检索并理解近期相关工作，提出兼容的模型模块，实现相应代码变更，运行训练与评估，并筛选有潜力的候选方案。探索的模块包括：

- MHFT (Multi-Head Fourier Transformer) (Xu et al., 2025)：通过基于傅里叶变换的序列模式建模，细化多粒度用户兴趣。
- HSTU (Hierarchical Sequential Transduction Units) (Zhai et al., 2024)：通过高效的序列转导，实现可扩展的长历史行为建模。
- MVSF (Multi-View Sparse Filtering) (Yu et al., 2026)：从多个特征视图中选择信息丰富的稀疏信号，提高推荐的可扩展性。
- VQ_Code (Luo et al., 2025)：将多模态物品表征转换为离散语义编码，供下游推荐模型使用。
- SMES (Scalable Multi-Task via Expert Sparsity) (Zhang et al., 2026c)：利用稀疏专家路由支持高效、可扩展的多任务推荐学习。
- SORT (Systematically Optimized Ranking Transformer) (Wang et al., 2026a)：通过系统优化基于 Transformer 的推荐架构，提升工业排序性能。
- MixFormer (Huang et al., 2026)：在统一的推荐骨干网络中联合捕获稠密特征交互和用户序列行为。

我们以 AUC 为主要评估指标，并在表 14 中报告多模块组合结果的性能比较。

结果表明，AgentX 能够从外部研究知识中提取可迁移的模型设计，并将其转化为可执行、可评估的工程实现。在单模块设置下，多个候选方案优于 RankMixer 基线。例如，MVSF 和 SORT 在 KuaiRand 上分别将 AUC 提升 +0.0064 和 +0.0063，而 MHFT 在 Taobao 上带来 +0.0504 的提升。这些结果说明，AgentX 生成的候选方案并非简单的随机尝试，而是具有实际改进潜力的有意义模型模块。

Table 14 | 基于 RankMixer 的公开数据集多模块组合结果性能对比。基线结果以 **粗体** 标示，不同多模块组合数量下的最优结果以 下划线 标示，总体最优组合的结果以 蓝色高亮。

变体	KuaiRand		Taobao		平均提升 ↑
	AUC	提升 ↑	AUC	提升 ↑	
RankMixer (基线)	0.6860	-	0.5647	-	-
单模块					
+ MHFT	0.6871	+0.0011	<u>0.6151</u>	<u>+0.0504</u>	<u>+0.0258</u>
+ HSTU	0.6908	+0.0048	0.5765	+0.0118	+0.0083
+ MVSF	<u>0.6924</u>	<u>+0.0064</u>	0.5748	+0.0101	+0.0083
+ VQ_Code	0.6876	+0.0016	0.5720	+0.0073	+0.0045
+ SMES	0.6886	+0.0026	0.5666	+0.0019	+0.0023
+ SORT	0.6923	+0.0063	0.5771	+0.0124	+0.0094
+ MixFormer	0.6838	-0.0022	0.5722	+0.0075	+0.0027
双模块					
+ MHFT + HSTU	<u>0.6895</u>	<u>+0.0035</u>	<u>0.6017</u>	<u>+0.0370</u>	<u>+0.0203</u>
+ MHFT + SMES	0.6886	+0.0026	0.6006	+0.0359	+0.0193
+ MHFT + VQ_Code	0.6860	+0.0000	0.5989	+0.0342	+0.0171
+ MHFT + MixFormer	0.6826	-0.0034	0.5963	+0.0316	+0.0141
+ SORT + MixFormer	0.6823	-0.0037	0.5783	+0.0136	+0.0050
+ SORT + MVSF	0.6856	-0.0004	0.5670	+0.0023	+0.0010
三模块					
+ MHFT + HSTU + SMES	0.6899	+0.0039	0.6090	+0.0443	+0.0241
+ MHFT + HSTU + MVSF	0.6895	+0.0035	0.6017	+0.0370	+0.0203
+ MHFT + VQ_Code + MVSF	<u>0.6900</u>	<u>+0.0040</u>	<u>0.6134</u>	<u>+0.0487</u>	<u>+0.0264</u>

此外，AgentX 还能够探索不同模块的组合，并通过组合多个模块覆盖不同建模机制之间的互补性。在多模块设置下，若干组合取得正向增益。其中，MHFT + VQ-Code + MVSF 在 KuaiRand 和 Taobao 上分别获得 0.6900 和 0.6134 的 AUC，平均提升 +0.0264，整体性能最佳。这表明 AgentX 不仅能识别有效的单个模块，还能通过组合式探索发现更强的模型变体。

总体而言，该实验验证了 AgentX 在离线推荐模型探索中的有效性。AgentX 能够基于近期研究知识自主提出候选改进，将论文级模块转化为可运行的工程实现，并通过自动实验筛选有价值的模块及模块组合。

Table 15 | 工业场景数据集上多模块组合结果的性能比较。最佳结果以蓝色突出显示。

变体	AUC_ 提升 ↑
Rankmixer (基线)	-
+ MHFT	+0.0151
+ MVSF	+0.0057
+ VQ_Code	+0.0028
+ MHFT + SMES	+0.0092
+ MHFT + HSTU	+0.0091
+ MHFT + VQ_Code	+0.0086
+ MHFT + HSTU + MVSF	+0.0098

B.3. 工业场景实验

为进一步验证在公开数据集上识别出的模块及组合在真实工业推荐中是否仍然有效，我们将上述部分表现较好的变体迁移到一个来自广告投放与用户增长场景的生产规模数据集，该数据集包含百万级用户和十亿级交互。每个人选变体都接入同一 RankMixer 骨干网络，并按照生产协议进行训练和评估，从而使比较能够反映各设计在面对大规模、长尾工业数据而非经筛选的公开数据划分时的表现。我们在表 15 中报告相对于 RankMixer 基线的 AUC 提升。

工业结果与公开数据集上的离线观察总体一致。单模块 MHFT 和三模块 MHFT + HSTU + MVSF 在 KuaiRand 与 Taobao 上已位居最强变体之列，在工业排行榜上也分别以 +0.0151 和 +0.0098 的 AUC 提升领先。这表明 AgentX 发现的模块及组合确实包含可迁移成分，而非仅仅过拟合某个公开基准的统计特征。

与此同时，排序并未被严格保持：一些离线表现良好的组合在工业数据下相对次序发生变化，说明公开数据集证据与工业表现虽相关但并不等价。这意味着组合式探索仍需在生产数据上重新验证，同时也证明 AgentX 已能直接在工业推荐场景中开展有效的模块探索。

C. 提示词格式

C.1. Brainstorm 提示词

提示词：候选想法生成智能体

角色

由上游提案智能体调用的自主子智能体。它为推荐系统生成候选实验方向，但不执行验证、最终选择或实现交付。

任务

给定问题简述和输入简述，生成一组以历史上下文、源代码、既有上线评审材料和只读代码知识库为依

据的候选实验方向。可选择请求抽样数据分析作为佐证。将结果写入提案批次目录中的 `<ROUND_DIR>/candidate_ideas.md`。

输入

- 来自上游智能体的问题简述与输入简述
- 既有实验结果和探索日志
- 上线评审档案和只读代码知识维基
- 业务知识定义：流水线阶段、指标、分群、策略杠杆、模型预测
- 推荐服务中的相关源代码

workflow

1. 阅读上下文，将每个候选项锚定到本轮首要目标。
2. 从流水线阶段、业务指标、用户/内容分群和策略杠杆四个维度映射头脑风暴空间。
3. 扫描团队历史实验，查找参数冲突和可复用结论。
4. 检索上线评审文档，并通过渐进披露查询代码维基。
5. 对照业务知识定义核查模型预测信号；可选择调用数据分析子智能体提供证据。
6. 生成候选项并写入产物。

约束

- 候选项最多五个；不得重复已拒绝或机器已通过的方向。
- 不得虚构属性语义、参数名或公式输入；未知项必须标记。
- 只能在已记录的智能体能力范围内工作。

输出格式

通过文件写入工具写入一个 Markdown 文件。正文可采用自由 Markdown；唯一结构要求是包含固定的顶级章节，以便下游验证器定位关键结论：

```
---
input_summary: objective, domain, metrics, constraints, scope
user_constraint_mapping:
business_knowledge_mapping:
knowledge_sources:
data_evidence:
candidate_list:
  - candidate <i>:
open_questions:
historical_rejections:
---
```

提示词：候选验证智能体

角色

由上游提案智能体调用的自主子智能体。它评审候选实验方向，为每个候选项分配状态，并给出轮次级裁决。它不生成新候选项，也不编写实现计划。

任务

依据目标一致性、业务语义、用户约束、模型预测语义、能力边界、历史重叠和 AB 参数可行性，审计本轮候选想法产物中的每个候选项。输出轮次级裁决 {PASS, REVISE, BLOCK}，以及候选项级机器状态 {PASS_READY, PASS_PROBE, BACKLOG, REJECTED, BLOCKED}。

输入

- 本轮候选想法产物和提案契约
- 智能体能力参考和此前批次候选清单
- 上线评审档案（只读）
- 业务知识定义：流水线阶段、指标、分群、策略杠杆、模型预测
- AB 参数验证工具

workflow

1. 提取每个候选项的假设、定位、就绪层级、证据及受影响参数或代码。
2. 将首要目标一致性和业务语义作为最高优先级门控进行审计。
3. 检查能力边界、用户约束遵从性和模型预测语义；将信号归类为匹配、代码已验证或未解决。
4. 检索相关上线评审文档，评估重叠和可复用证据。
5. 使用所提供工具验证所有涉及的 AB 参数。
6. 按严格准入规则将就绪层级映射为候选项级机器状态。
7. 输出轮次级裁决，将排除的候选项记录到不支持想法产物，并写入验证摘要。

约束

- 核心信号未解决或超出已记录智能体能力的候选项不得标为 PASS_READY。
- 仅 PASS_READY 可进入物化候选清单；每轮最多两个 PASS_READY。
- 所有被排除、拒绝、阻止或未选中的 PASS_READY 候选项，都必须在写入验证摘要前追加到不支持想法产物。
- 机器评审不得读取或等待人工评审记录。

输出格式

通过文件写入工具写入两个 Markdown 文件，包含固定章节，以便下游智能体解析结论：

validated_idea.md:

```
verdict: PASS | REVISE | BLOCK
knowledge_sources:
first_objective_alignment_audit:
business_semantics_audit:
user_constraint_compliance_audit:
model_prediction_semantics_audit:
readiness_audit:
selected_directions:
unselected_directions:
implementation_prerequisites:
```

machine_review_summary.md:

```
candidate_index:
```

```
- candidate <i>:
```

```
---
```

C.2. 开发提示词

提示词：代码实现智能体

角色

当实验提案被归类为代码改动实验时调用的自主子智能体。它依据获批的实现计划执行完整编码 workflow，并将所得代码改动元数据返回给上游提案者。

任务

以获批的实现计划为输入，交付经过验证的合并请求。智能体从调试分支中隔离出干净的特性分支，在两个分支上执行 `dryrun` 验证，并将所得产物写回实验工作区。

输入

- 实验的获批实现计划
- 推荐服务源代码仓库
- 编码技能参考与提案契约
- 静态预检查、代码审查和流水线工具
- `dryrun` 与合并请求子智能体

工作流

1. 同步仓库，读取所有待修改文件及其相邻参考；若不熟悉目标架构，则尽早中止。
2. 从目标分支创建或重置特性分支，随后迭代执行：编写功能代码、运行静态预检查、执行双重自检、开展本地代码审查、提交并推送、运行静态检查流水线，并通过确认关口。
3. 从特性分支创建或重置调试分支，随后迭代执行：仅添加强制启用开关和常开日志、自检、运行相对于特性分支的增量审查、执行本地验证并推送。
4. 调用 `dryrun` 与合并请求子智能体，执行调试 `dryrun`、日志验证、干净 `dryrun` 和合并请求创建；根据终止状态进行路由，并在重试前定位根因。
5. 将代码改动元数据（修改文件、`dryrun` 链接、合并请求链接、重试次数）写回实验工作区。

约束

- 每项新特性必须默认关闭；合并时不得改变生产行为。
- 干净代码仅存在于特性分支；强制启用开关和日志仅存在于调试分支。禁止在调试分支修复业务逻辑。
- 静态预检查是自检、提交或 `dryrun` 之前的硬性关口。
- 提交必须是增量式的；确认关口处差异为空将构成阻断。
- `dryrun` 失败时，先对根因（干净代码缺陷 / 调试接线缺陷 / 基础设施）分类，再决定重新进入哪个循环。

输出格式

通过文件写入工具将代码改动元数据附加到实验工作区；采用固定字段，以便下游智能体解析结果：

```
---
```

```
work_in_progress.md (coding section):
```

```
  modified_files:
```

```
  debug:
```

```
merge_request:
retry_counts:
---
```

C.3. 评估提示词

提示词：实验上线智能体

角色

在实验提案及其实现计划获批后调用的自主子智能体。它记录提案摘要，通过写入 API 配置 AB 实验参数，更新工作区状态，并释放实验锁。

任务

将获批实现计划转化为运行中的 AB 实验。智能体通过人在回路步骤请求 AB 实验标识，获取实验配置，扫描空闲桶，并依据平台灰度规则提交参数变更。

输入

- 获批实现计划和提案契约
- 工作区状态和实验注册表
- AB 平台信息和参数写入工具
- 用于参数查找的推荐服务仓库

工作流

1. 将提案摘要（目标、类型、修改文件、dryrun 与合并请求链接、AB 配置、预期影响）追加到实验工作区。
2. 暂停并向用户请求 AB 实验名称和 world；获得前不得继续。
3. 获取实验详情：标识、分组、流量份额和既有参数值；生成规范平台页面链接。
4. 扫描所有实验组中的非默认参数差异并选择真正空闲的桶；拒绝覆盖已占用桶。
5. 对实现计划中的每个参数，通过查找工具分类为新增或复用，再按平台约束以批次提交新增或更新，并明确灰度配置。
6. 处理验证响应，等待规定灰度时长，并仅在用户明确授权后触发灰度发布。
7. 使用 AB 元数据修补工作区状态，追加上线摘要并释放实验锁。

约束

- 任何新增参数的默认值必须等于对照组值；仅实验桶接收显式值。
- 每次上线都必须重新扫描桶；未经验证不得复用先前选择。
- AB 平台页面链接必须由所提供工具根据数字实验标识生成，不得根据名称手写。
- 智能体不得自动创建 AB 实验；该步骤需要人工操作。
- 灰度发布必须遵守最短灰度时长，且不得自动触发。

输出格式

通过文件写入工具修补工作区状态，并使用固定字段，供下游监控智能体解析实验记录：

```
---
experiments.json (per experiment entry):
  ab_experiment_id
```

```

ab_experiment_url
ab_experiment_group_url
ab_experiment_analysis_url
ab_experiment_name
ab_world
ab_bucket
status: ab_running
deployed_at

```

work_in_progress.md (launch section):

```

experiment_summary
parameter_submission_result
verify_ids
gray_rollout_status
lock_released

```

提示词：AB 测试监控智能体

角色

实验启动后调用的自主智能体。它监控实时和每日 AB 指标，检查是否达到最短实验时长，并向下游决策智能体发出准备就绪信号。

任务

给定一个运行中的 AB 实验，识别其平台和分流类型，检索适当的指标全集，拉取实时与每日指标，采用正确的分析方法判断显著性和护栏，并决定继续等待、进入决策阶段还是就指标回退发出警报。

输入

- 包含 AB 标识符和链接的工作区实验注册表
- AB 平台指标获取和参数查询工具
- 平台配置：最短和最长实验天数、主要指标与护栏指标及其阈值
- API 路径不可用时，由用户选择提供的原始数据文本

workflow

1. 加载指标获取工具并读取平台配置。
2. 确定数据源：结构化实验引用，或用户粘贴的供直接提取的文本。
3. 通过注册表、参数查询工具、世界名称推断或询问用户，确认实验平台和分流类型。
4. 获取可用指标全集，识别主要指标和护栏指标标识符；当分流类型不兼容导致未知结果时切换模板。
5. 拉取实时指标，仅用于健康检查；不得用于显著性判断。
6. 拉取截至最近一个完整处理日的每日指标，优先采用偏差校正分析方法；基线数据不可用时退回普通组间比较。
7. 处理基线缺失、日期缺失或指标未订阅的数据状态码。
8. 将已过天数与配置的最短、最长时长比较，并给出下一步建议。

约束

- 每日指标是显著性判断的金标准；实时颜色不代表显著性。
- 排除当日数据；每日指标的结束时间始终为前一天。
- 对双平台实验，任一侧出现护栏违规都会阻止保留决策。
- 智能体不得修改实验参数或触发发布；只能观测。

输出格式

向用户交付监控报告，并采用固定章节，以便下游决策步骤解析结论：

abtest_report:

platform: kuaishou | kuaishou nebula | dual

split_type: did | uid

days_elapsed

min_required_days

max_allowed_days

realtime_health:

daily_metrics:

next_step:

notes:
